

Projet_4

December 22, 2025

```
[1]: # Etape 1 - Importation des librairies et chargement des fichiers
```

```
[2]: # 1.1 - Importation des librairies
```

```
[3]: import pandas as pd
from IPython.display import display
import matplotlib.pyplot as plt
from matplotlib import colors
import plotly.graph_objects as go
import numpy as np
```

```
[4]: # 1.2 - Chargement des fichiers
population = pd.read_csv('population.csv')
sous_nutrition = pd.read_csv('sous_nutrition.csv')
dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv')
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')
```

```
[5]: # .Visualiser 5 premières lignes fichier population
print("Population :")
display(population.head())
```

Population :

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
[6]: # .Visualiser 5 premières lignes fichier sous_nutrition
print("Sous_nutrition :")
display(sous_nutrition.head())
```

Sous_nutrition :

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9

```
3 Afghanistan 2015-2017 9.7
4 Afghanistan 2016-2018 10.5
```

```
[7]: # .Visualiser 5 premières lignes fichier dispo_alimentaire
print("Disponibilité alimentaire :")
display(dispo_alimentaire.head())
```

Disponibilité alimentaire :

	Zone	Produit	Origine	Aliments pour animaux \
0	Afghanistan	Abats Comestible	animale	NaN
1	Afghanistan	Agrumes, Autres	vegetale	NaN
2	Afghanistan	Aliments pour enfants	vegetale	NaN
3	Afghanistan	Ananas	vegetale	NaN
4	Afghanistan	Bananes	vegetale	NaN

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
0	NaN	5.0
1	NaN	1.0
2	NaN	1.0
3	NaN	0.0
4	NaN	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
0	1.72
1	1.29
2	0.06
3	0.00
4	2.70

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	NaN
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	NaN
4	0.05

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53.0	NaN	NaN
1	41.0	2.0	40.0
2	2.0	NaN	2.0
3	0.0	NaN	0.0

4		82.0		NaN		82.0
	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53.0	NaN	53.0	NaN	NaN	NaN
1	39.0	2.0	3.0	NaN	NaN	NaN
2	2.0	NaN	NaN	NaN	NaN	NaN
3	0.0	NaN	NaN	NaN	NaN	NaN
4	82.0	NaN	NaN	NaN	NaN	NaN

```
[8]: # .Visualiser 5 premières lignes fichier aide_alimentaire
print("Aide alimentaire :")
display(aide_alimentaire.head())
```

Aide alimentaire :

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
[9]: # Etape 2 - Analyse exploratoire des fichiers
```

```
[10]: # 2.1 - Analyse exploratoire du fichier population
```

```
[11]: # Afficher les dimensions du dataset
nb_lignes, nb_colonnes = population.shape
print(f"Le tableau comporte {nb_lignes} lignes (observations).")
print(f"Le tableau comporte {nb_colonnes} colonnes.\n")
```

Le tableau comporte 1416 lignes (observations).
Le tableau comporte 3 colonnes.

```
[12]: # La nature des données dans chacune des colonnes
print("Nature des données dans chaque colonne :")
print(population.dtypes)
```

Nature des données dans chaque colonne :

```
Zone      object
Année     int64
Valeur    float64
dtype: object
```

```
[13]: # Le nombre de valeurs présentes dans chacune des colonnes POPULATION (si une
      ↪ cellule contient des cases vides le nb sera < à 1416
print("Nombre de valeurs non nulles dans chaque colonne :")
print(population.count())
```

Nombre de valeurs non nulles dans chaque colonne :

Zone 1416

Année 1416

Valeur 1416

dtype: int64

```
[14]: # Afficher les 5 premières lignes
population.head()
```

```
[14]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
[15]: # Multiplier la colonne Valeur par 1000 pour harmoniser les unités - POPULATION
population['Valeur'] = population['Valeur'] * 1000
```

```
[16]: # Vérification rapide de l'harmonisation des unités - POPULATION
population.head()
```

```
[16]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

```
[17]: # changement du nom de la colonne Valeur par Population - Population
population = population.rename(columns={'Valeur': 'Population'})
```

```
[18]: # Vérification du changement de nom de la colonne - POPULATION
print(population.columns)
```

```
Index(['Zone', 'Année', 'Population'], dtype='object')
```

```
[19]: # Afficher les 5 premières lignes
population.head()
```

```
[19]:
```

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

```
[20]: # 2.2 - Analyse exploratoire du fichier disponibilité alimentaire
```

```
[21]: # Afficher les dimensions du dataset DISPONIBILITE ALIMENTAIRE
nb_lignes, nb_colonnes = dispo_alimentaire.shape
print(f"Le tableau comporte {nb_lignes} observation(s) ou article(s)")
print(f"Le tableau comporte {nb_colonnes} colonne(s)")
```

Le tableau comporte 15605 observation(s) ou article(s)
 Le tableau comporte 18 colonne(s)

```
[22]: # Demander le nombre de ligne DISPONIBILITE ALIMENTAIRE
print(f"Le tableau comporte {len(dispo_alimentaire)} ligne(s)")
```

Le tableau comporte 15605 ligne(s)

```
[23]: # Nature des données - DISPONIBILITE ALIMENTAIRE
print("Nature des données dans chaque colonne :")
print(dispo_alimentaire.dtypes)
```

```
Nature des données dans chaque colonne :
Zone                                object
Produit                            object
Origine                            object
Aliments pour animaux              float64
Autres Utilisations                 float64
Disponibilité alimentaire (Kcal/personne/jour) float64
Disponibilité alimentaire en quantité (kg/personne/an) float64
Disponibilité de matière grasse en quantité (g/personne/jour) float64
Disponibilité de protéines en quantité (g/personne/jour) float64
Disponibilité intérieure            float64
Exportations - Quantité             float64
Importations - Quantité             float64
Nourriture                         float64
Pertes                             float64
Production                         float64
Semences                           float64
Traitement                         float64
Variation de stock                  float64
dtype: object
```

```
[24]: # Affichage les 5 premières lignes de la table - DISPONIBILITE ALIMENTAIRE
display(dispo_alimentaire.head(5))
```

	Zone	Produit	Origine	Aliments pour animaux \
0	Afghanistan	Abats Comestible	animale	NaN
1	Afghanistan	Agrumes, Autres	vegetale	NaN
2	Afghanistan	Aliments pour enfants	vegetale	NaN
3	Afghanistan	Ananas	vegetale	NaN
4	Afghanistan	Bananes	vegetale	NaN

Autres Utilisations Disponibilité alimentaire (Kcal/personne/jour) \

0	NaN	5.0
1	NaN	1.0
2	NaN	1.0
3	NaN	0.0
4	NaN	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
0	1.72
1	1.29
2	0.06
3	0.00
4	2.70

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	NaN
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	NaN
4	0.05

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53.0	NaN	NaN
1	41.0	2.0	40.0
2	2.0	NaN	2.0
3	0.0	NaN	0.0
4	82.0	NaN	82.0

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53.0	NaN	53.0	NaN	NaN	NaN
1	39.0	2.0	3.0	NaN	NaN	NaN
2	2.0	NaN	NaN	NaN	NaN	NaN
3	0.0	NaN	NaN	NaN	NaN	NaN
4	82.0	NaN	NaN	NaN	NaN	NaN

```
[25]: # remplacement des NaN dans le dataset par des 0 - DISPONIBILITE ALIMENTAIRE
dispo_alimentaire = dispo_alimentaire.fillna(0)
```

```
[26]: # Vérification : afficher les 5 premières lignes - DISPONIBILITE ALIMENTAIRE
dispo_alimentaire.head()
```

[26] :

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale		0.0
1	Afghanistan	Agrumes, Autres	vegetale		0.0
2	Afghanistan	Aliments pour enfants	vegetale		0.0
3	Afghanistan	Ananas	vegetale		0.0
4	Afghanistan	Bananes	vegetale		0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	0.0		5.0
1	0.0		1.0
2	0.0		1.0
3	0.0		0.0
4	0.0		4.0

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
0	0.20	
1	0.01	
2	0.01	
3	0.00	
4	0.02	

	Disponibilité de protéines en quantité (g/personne/jour)	\
0	0.77	
1	0.02	
2	0.03	
3	0.00	
4	0.05	

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	\
0	53.0	0.0	0.0	
1	41.0	2.0	40.0	
2	2.0	0.0	2.0	
3	0.0	0.0	0.0	
4	82.0	0.0	82.0	

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53.0	0.0	53.0	0.0	0.0	0.0
1	39.0	2.0	3.0	0.0	0.0	0.0
2	2.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0

4	82.0	0.0	0.0	0.0	0.0	0.0
---	------	-----	-----	-----	-----	-----

```
[27]: # multiplication de toutes les lignes contenant des milliers de tonnes en Kg -
↳- DISPONIBILITE ALIMENTAIRE
# Création d'une liste en selectionnant les colonnes concernées par la
↳multiplicat°
colonnes_tonnes = [
    'Aliments pour animaux',
    'Autres Utilisations',
    'Disponibilité intérieure',
    'Exportations - Quantité',
    'Importations - Quantité',
    'Nourriture',
    'Pertes',
    'Production',
    'Semences',
    'Traitement',
    'Variation de stock'
]

# Multiplier la liste par 1 000 000 pour passer en kg
dispo_alimentaire[colonnes_tonnes] = dispo_alimentaire[colonnes_tonnes] *
↳1_000_000
```

```
[28]: # Vérification : afficher les 5 premières lignes - DISPONIBILITE ALIMENTAIRE
display(dispo_alimentaire.head(5))
```

	Zone	Produit	Origine	Aliments pour animaux \
0	Afghanistan	Abats Comestible	animale	0.0
1	Afghanistan	Agrumes, Autres	vegetale	0.0
2	Afghanistan	Aliments pour enfants	vegetale	0.0
3	Afghanistan	Ananas	vegetale	0.0
4	Afghanistan	Bananes	vegetale	0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
0	0.0	5.0
1	0.0	1.0
2	0.0	1.0
3	0.0	0.0
4	0.0	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
0	1.72
1	1.29
2	0.06
3	0.00
4	2.70

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	0.00
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	0.00
4	0.05

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53000000.0	0.0	0.0
1	41000000.0	2000000.0	40000000.0
2	2000000.0	0.0	2000000.0
3	0.0	0.0	0.0
4	82000000.0	0.0	82000000.0

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53000000.0	0.0	53000000.0	0.0	0.0	0.0
1	39000000.0	2000000.0	3000000.0	0.0	0.0	0.0
2	2000000.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	82000000.0	0.0	0.0	0.0	0.0	0.0

```
[29]: # 2.3 - Analyse exploratoire du fichier aide alimentaire
```

```
[30]: # Afficher les dimensions du dataset - AIDE ALIMENTAIRE
nb_lignes, nb_colonnes = aide_alimentaire.shape
print(f"Le tableau comporte {nb_lignes} observation(s) ou article(s)")
print(f"Le tableau comporte {nb_colonnes} colonne(s)")
```

Le tableau comporte 1475 observation(s) ou article(s)
Le tableau comporte 4 colonne(s)

```
[31]: # Demander le nombre de ligne - AIDE ALIMENTAIRE
print(f"Le tableau comporte {len(aide_alimentaire)} ligne(s)")
```

Le tableau comporte 1475 ligne(s)

```
[32]: # Nature des données - AIDE ALIMENTAIRE
print("Nature des données dans chaque colonne :")
print(aide_alimentaire.dtypes)
```

Nature des données dans chaque colonne :
Pays bénéficiaire object

```
Année          int64
Produit        object
Valeur         int64
dtype: object
```

```
[33]: # Vérification : afficher les 5 premières lignes - AIDE ALIMENTAIRE
aide_alimentaire.head()
```

```
[33]: Pays bénéficiaire  Année      Produit  Valeur
0      Afghanistan    2013  Autres non-céréales    682
1      Afghanistan    2014  Autres non-céréales    335
2      Afghanistan    2013      Blé et Farin   39224
3      Afghanistan    2014      Blé et Farin   15160
4      Afghanistan    2013      Céréales    40504
```

```
[34]: # changement du nom de la colonne Pays bénéficiaire par Zone - AIDE ALIMENTAIRE
aide_alimentaire = aide_alimentaire.rename(columns={'Pays bénéficiaire':
↪ 'Zone'})
```

```
[35]: # Vérification : afficher les 5 premières lignes - AIDE ALIMENTAIRE
aide_alimentaire.head()
```

```
[35]:      Zone  Année      Produit  Valeur
0  Afghanistan    2013  Autres non-céréales    682
1  Afghanistan    2014  Autres non-céréales    335
2  Afghanistan    2013      Blé et Farin   39224
3  Afghanistan    2014      Blé et Farin   15160
4  Afghanistan    2013      Céréales    40504
```

```
[36]: # Multiplication de la colonne Aide_alimentaire qui contient des tonnes par
↪ 1000 pour avoir des kg - AIDE ALIMENTAIRE
aide_alimentaire['Valeur'] = aide_alimentaire['Valeur'] * 1000
```

```
[37]: # Vérification : afficher les 5 premières lignes - AIDE ALIMENTAIRE
aide_alimentaire.head()
```

```
[37]:      Zone  Année      Produit  Valeur
0  Afghanistan    2013  Autres non-céréales   682000
1  Afghanistan    2014  Autres non-céréales   335000
2  Afghanistan    2013      Blé et Farin  39224000
3  Afghanistan    2014      Blé et Farin  15160000
4  Afghanistan    2013      Céréales   40504000
```

```
[38]: # 2.3 - Analyse exploratoire du fichier SOUS NUTRITION
```

```
[39]: # Afficher les dimensions du dataset - SOUS NUTRITION
nb_lignes, nb_colonnes = sous_nutrition.shape
print(f"Le tableau comporte {nb_lignes} observation(s) ou article(s)")
```

```
print(f"Le tableau comporte {nb_colonnes} colonne(s)")
```

Le tableau comporte 1218 observation(s) ou article(s)

Le tableau comporte 3 colonne(s)

```
[40]: # Nature des données - SOUS NUTRITION
print("Nature des données dans chaque colonne :")
print(sous_nutrition.dtypes)
```

Nature des données dans chaque colonne :

Zone object

Année object

Valeur object

dtype: object

```
[41]: # afficher les 5 premières lignes - SOUS NUTRITION
sous_nutrition.head()
```

```
[41]:
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
[42]: # #Conversion de la colonne sous nutrition en numérique
# sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'])
# avec l'argument errors=coerce qui permet de convertir automatiquement les
# ↪ lignes qui ne sont pas des nombres en NaN) - SOUS NUTRITION

sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'],
# ↪ errors='coerce')

# Remplacer les NaN par 0 - SOUS NUTRITION
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].fillna(0)
```

```
[43]: # Afficher les lignes 56 à 69 (incluses)- SOUS NUTRITION
sous_nutrition.iloc[56:70]
```

```
[43]:
```

	Zone	Année	Valeur
56	Argentine	2014-2016	1.1
57	Argentine	2015-2017	1.3
58	Argentine	2016-2018	1.5
59	Argentine	2017-2019	1.7
60	Arménie	2012-2014	0.0
61	Arménie	2013-2015	0.0
62	Arménie	2014-2016	0.0
63	Arménie	2015-2017	0.0

64	Arménie	2016-2018	0.0
65	Arménie	2017-2019	0.0
66	Australie	2012-2014	0.0
67	Australie	2013-2015	0.0
68	Australie	2014-2016	0.0
69	Australie	2015-2017	0.0

```
[44]: # Changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition = sous_nutrition.rename(columns={'Valeur': 'sous_nutrition'})
```

```
[45]: # Multiplication de la colonne sous_nutrition en 1 000 000 - SOUS NUTRITION
sous_nutrition['sous_nutrition'] = sous_nutrition['sous_nutrition'] * 1000000
```

```
[46]: # Vérification : afficher les 5 premières lignes - SOUS NUTRITION
sous_nutrition.head()
```

```
[46]:
```

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

```
[47]: # . 3.1 - Proportion de personnes en sous nutrition
```

```
[48]: # Il faut tout d'abord faire une jointure entre la table population et la
      ↪table sous nutrition, en ciblant l'année 2017

# Filtrer l'année 2017 pour la population
population_2017 = population[population['Année'] == 2017]

# Filtrer la période de sous-nutrition qui contient 2017
sous_nutrition_2017 = sous_nutrition[sous_nutrition['Année'] == '2016-2018']

# # Puis joindre les 2 tables Population + Sous nutrition_ Utiliser left join
      ↪pour conserver tous les pays du fichier population
resultat = population_2017.merge(sous_nutrition_2017, on="Zone", how="left")
```

```
[49]: # Affichage du dataset du résultat
print(resultat.head())
```

	Zone	Année_x	Population	Année_y	sous_nutrition
0	Afghanistan	2017	36296113.0	2016-2018	10500000.0
1	Afrique du Sud	2017	57009756.0	2016-2018	3100000.0
2	Albanie	2017	2884169.0	2016-2018	100000.0
3	Algérie	2017	41389189.0	2016-2018	1300000.0
4	Allemagne	2017	82658409.0	2016-2018	0.0

```
[50]: # Calcul et affichage du nombre de personnes en état de sous nutrition en 2017
total_sous_nutrition = resultat['sous_nutrition'].sum()

print(f"Nombre total de personnes en état de sous-nutrition en 2017 :␣
↪{total_sous_nutrition:,.0f}")
```

Nombre total de personnes en état de sous-nutrition en 2017 : 535,700,000

```
[51]: # Pour une meilleur interpretation/visualiation du chiffre mettre en %

# Calcul de la population mondiale totale en 2017
population_mondiale_2017 = resultat['Population'].sum()

# Calcul du pourcentage de personnes en sous-nutrition = (ss nutrition total/␣
↪population mondiale)*100
pourcentage_sous_nutrition = (total_sous_nutrition / population_mondiale_2017)␣
↪* 100
print(f"Pourcentage de personnes en état de sous-nutrition en 2017 :␣
↪{pourcentage_sous_nutrition:.2f}%")
```

Pourcentage de personnes en état de sous-nutrition en 2017 : 7.10%

```
[52]: population_mondiale_2017 = resultat['Population'].sum()
print(f"population_mondiale_2017 : {population_mondiale_2017:.0f}")
```

population_mondiale_2017 : 7548134111

```
[53]: # Préparation visuel d'un graphique type camembert

# Préparation des valeurs pour le camembert
Population_en_sous_nutrition = total_sous_nutrition
Population_en_securite_alimentaire = population_mondiale_2017 -␣
↪total_sous_nutrition

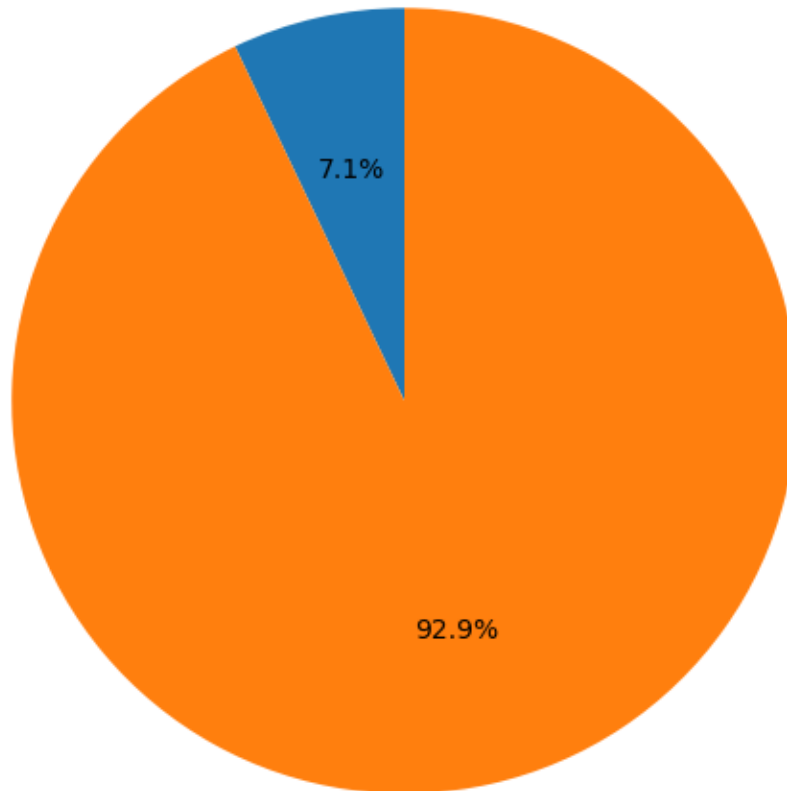
labels = ['Population en sous-nutrition', 'Population en sécurité alimentaire']
valeurs = [Population_en_sous_nutrition, Population_en_securite_alimentaire]

# Création du camembert
plt.figure(figsize=(6, 6))
plt.pie(
    valeurs,
    labels=labels,
    autopct='%1.1f%%', # affiche les % avec 1 décimale
    startangle=90      # commence à 90° pour un affichage plus lisible
)
plt.title("Proportion de la sous-nutrition dans la population mondiale (2017)")
plt.tight_layout()
```

```
plt.show()
```

Proportion de la sous-nutrition dans la population mondiale (2017)

Population en sous-nutrition



Population en sécurité alimentaire

```
[54]: # REQUETE SUPPLEMENTAIRE - EVOLUTION DU NOMBRE DE PERSONNE EN ETAT DE SOUS-  
      ↪NUTRITION DE 2013 A 2017
```

```
[55]: # Filtrer les données de population et sous_nutrition  
      # Emploi de isin/copy => si [population["Année"]] garde slt si vrai et fait  
      ↪une vrai copie pr éviter pb  
population_filtre = population[population["Année"].isin([2013, 2014, 2015,  
      ↪2016, 2017])].copy()  
sous_nutrition_filtre = sous_nutrition[sous_nutrition["Année"].  
      ↪isin(["2012-2014", "2013-2015", "2014-2016", "2015-2017", "2016-2018"])].  
      ↪copy()
```

```

# Convertir les périodes en années (année centrale)
correspondance = {
    "2012-2014": 2013,
    "2013-2015": 2014,
    "2014-2016": 2015,
    "2015-2017": 2016,
    "2016-2018": 2017
}
sous_nutrition_filtre["Année"] = sous_nutrition_filtre["Année"].
↳map(correspondance)

# Fusionner population et sous-nutrition
df_merged = population_filtre.merge(sous_nutrition_filtre, on=["Zone", "Année"])

# Grouper ttes les lignes ~m année et additionne population et ss nutrition
taux_mondial = df_merged.groupby("Année").agg(population_totale=("Population",
↳"sum"),sous_nutrition_totale=("sous_nutrition", "sum")).reset_index()

# Calcul du taux (proportion) puis du pourcentage
taux_mondial["Taux_sous_nutrition"] = (taux_mondial["sous_nutrition_totale"] /
↳taux_mondial["population_totale"])
taux_mondial["TX_SS_Nutrit"] = taux_mondial["Taux_sous_nutrition"] * 100

print(taux_mondial)

```

	Année	population_totale	sous_nutrition_totale	Taux_sous_nutrition \
0	2013	7.206670e+09	528100000.0	0.073279
1	2014	7.291347e+09	523500000.0	0.071797
2	2015	7.375818e+09	524700000.0	0.071138
3	2016	7.460004e+09	528600000.0	0.070858
4	2017	7.543799e+09	535700000.0	0.071012

	TX_SS_Nutrit
0	7.327934
1	7.179743
2	7.113787
3	7.085787
4	7.101197

[56]:

```

# Faire copie pour éviter modif de l'original
taux_mondial_formate = taux_mondial.copy()

# Modificat° données population pour meilleur visuel:séparateur de milliers,
↳transformat°nbr en texte (9e+09)
taux_mondial_formate["population_totale"] =
↳taux_mondial_formate["population_totale"].apply(lambda x: f"{x:,.0f}").
↳replace(",", " ")

```

```

# Modificat° données sous-nutrition
taux_mondial_format["sous_nutrition_totale"] =
    ↪taux_mondial_format["sous_nutrition_totale"].apply(lambda x: f"{x:,.0f}").
    ↪replace(",", " ")

# Mettre le taux en pourcentage avec 2 décimales
taux_mondial_format["TX_SS_Nutrit"] = taux_mondial_format["TX_SS_Nutrit"].
    ↪apply(lambda x: f"{x:.2f} %")

# Conserver uniquement les colonnes utiles
taux_mondial_format = taux_mondial_format[["Année", "population_totale",
    ↪"sous_nutrition_totale", "TX_SS_Nutrit"]]

# Faire tableau stylé pr power point
styled_table = (
    taux_mondial_format.style
    .set_caption("Évolution mondiale de la sous-nutrition")
    .set_table_styles([
        {'selector': 'caption', 'props': [('font-size', '18px'),
    ↪('font-weight', 'bold'), ('text-align', 'left')]},
        {'selector': 'th', 'props': [('background-color', '#2C3E50'), ('color',
    ↪'white'), ('padding', '8px')]},
        {'selector': 'td', 'props': [('padding', '6px')]}
    ])
    .hide(axis="index")
)
styled_table

```

[56]: <pandas.io.formats.style.Styler at 0x146ee5160>

```

[57]: # Conversion en pourcentage pour le graphique
taux_percent = taux_mondial["Taux_sous_nutrition"] * 100

# faire graphique courbe => plot
plt.figure(figsize=(8, 5))
plt.plot(
    taux_mondial["Année"], taux_percent,
    marker='o', color='teal', linewidth=2
)

plt.title("Évolution du taux mondial de sous-nutrition (2013-2017)",
    ↪fontsize=13, pad=15)
plt.xlabel("Année", fontsize=11)
plt.ylabel("Taux de sous-nutrition (%)", fontsize=11)

# Ajouter les valeurs au-dessus des points

```

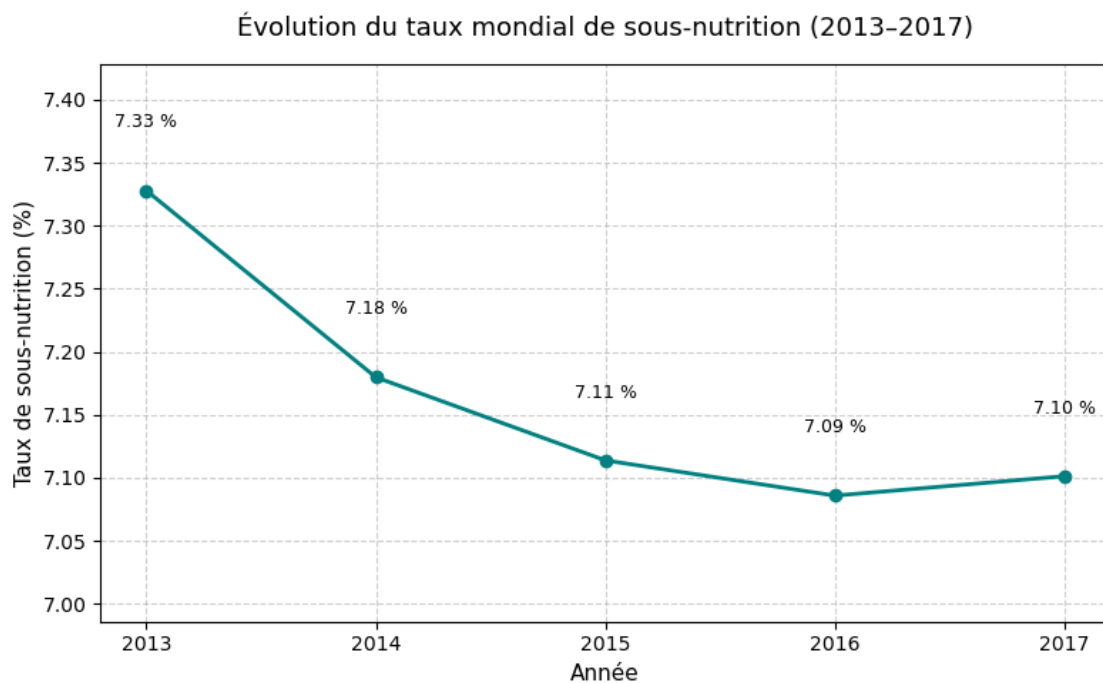


```

for i, val in enumerate(taux_percent):
    plt.text(
        taux_mondial["Année"].iloc[i],
        val + 0.05,
        f"{val:.2f} %",
        ha='center', fontsize=9, color='black'
    )

plt.grid(True, linestyle='--', alpha=0.6)
plt.xticks([2013, 2014, 2015, 2016, 2017])
plt.ylim(min(taux_percent) - 0.1, max(taux_percent) + 0.1)
plt.tight_layout()
plt.show()

```



```

[58]: plt.figure(figsize=(8, 5))
plt.plot(
    taux_mondial["Année"], taux_mondial["population_totale"],
    marker='o', linewidth=2
)

plt.title("Évolution de la population mondiale (2013-2017)", fontsize=13,
        pad=15)
plt.xlabel("Année", fontsize=11)
plt.ylabel("Population totale", fontsize=11)

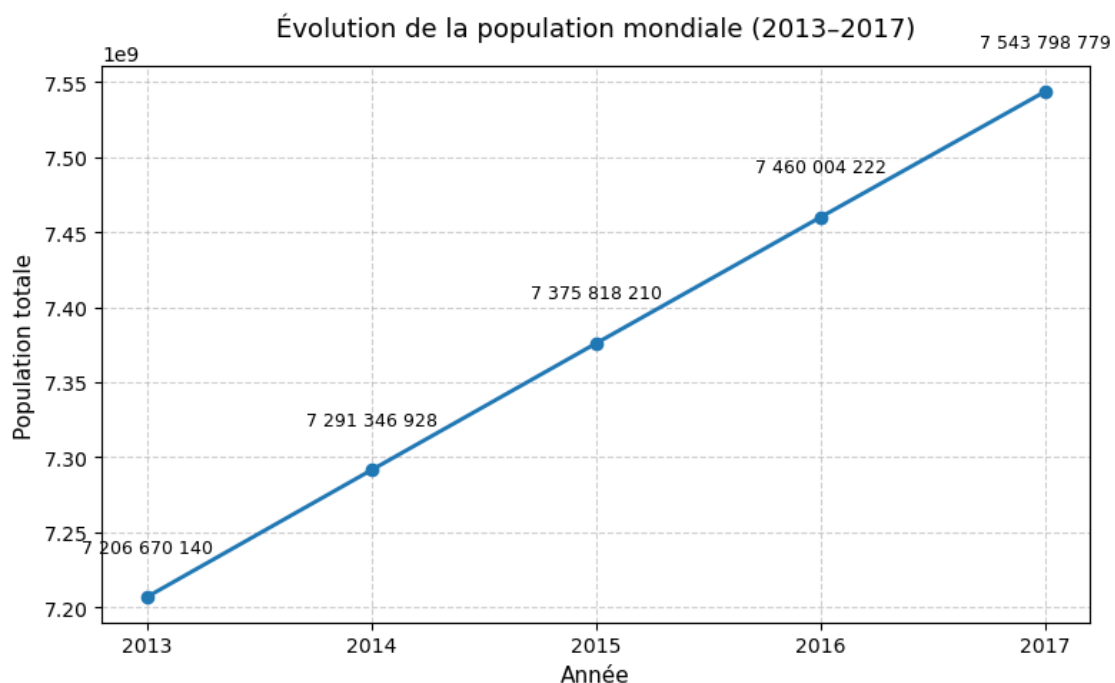
```

```

for i, val in enumerate(taux_mondial["population_totale"]):
    plt.text(
        taux_mondial["Année"].iloc[i],
        val + 30_000_000,
        f"{val:,.0f}".replace(",", " "),
        ha='center', fontsize=9
    )

plt.grid(True, linestyle='--', alpha=0.6)
plt.xticks([2013, 2014, 2015, 2016, 2017])
plt.tight_layout()
plt.show()

```



```

[59]: # Conversion en pourcentage
taux_percent = taux_mondial["Taux_sous_nutrition"] * 100

plt.figure(figsize=(12, 7))

# --- 1) Axe principal : taux de sous-nutrition (%)
ax1 = plt.gca()
line = ax1.plot(
    taux_mondial["Année"], taux_percent,
    marker='o', linewidth=3, label="Taux de sous-nutrition (%)",

```

```

        color='red', markersize=8
    )
    ax1.set_xlabel("Année", fontsize=12, fontweight='bold')
    ax1.set_ylabel("Taux de sous-nutrition (%)", fontsize=12, fontweight='bold',
        ↪color='red')
    ax1.tick_params(axis='y', labelcolor='red')
    ax1.grid(True, linestyle='--', alpha=0.5)

    # Ajuster les limites pour la courbe de sous-nutrition
    ax1.set_ylim(min(taux_percent) - 0.2, max(taux_percent) + 0.2)

    # Ajouter les valeurs pour le taux de sous-nutrition
    for i, val in enumerate(taux_percent):
        ax1.text(
            taux_mondial["Année"].iloc[i],
            val + 0.1,
            f"{val:.2f}%",
            ha='center', fontsize=10, fontweight='bold', color='red',
            bbox=dict(boxstyle="round,pad=0.3", facecolor="white", edgecolor='red',
            ↪alpha=0.8)
        )

    # --- 2) Axe secondaire : population mondiale en barres
    ax2 = ax1.twinx()

    # Convertir la population en milliards
    population_milliards = taux_mondial["population_totale"] / 1_000_000_000

    # Calculer les limites pour l'axe de population (pour voir les variations)
    pop_min = population_milliards.min()
    pop_max = population_milliards.max()
    pop_range = pop_max - pop_min

    # Commencer l'échelle plus haut pour voir les différences
    ax2.set_ylim(pop_min - 0.1, pop_max + 0.1) # Juste en dessous du minimum

    bars = ax2.bar(
        taux_mondial["Année"], population_milliards,
        alpha=0.7, color='steelblue', width=0.5,
        label="Population mondiale (milliards)"
    )
    ax2.set_ylabel("Population (milliards)", fontsize=12, fontweight='bold',
        ↪color='steelblue')
    ax2.tick_params(axis='y', labelcolor='steelblue')

    # Ajouter valeurs population sur les barres
    for i, val in enumerate(population_milliards):

```

```

ax2.text(
    taux_mondial["Année"].iloc[i],
    val + 0.01, # Très proche du sommet de la barre
    f"{val:.3f} Md", # 3 décimales pour voir les petites variations
    ha='center', fontsize=9, fontweight='bold',
    bbox=dict(boxstyle="round,pad=0.2", facecolor="white",
    ↪edgecolor='steelblue', alpha=0.9)
)

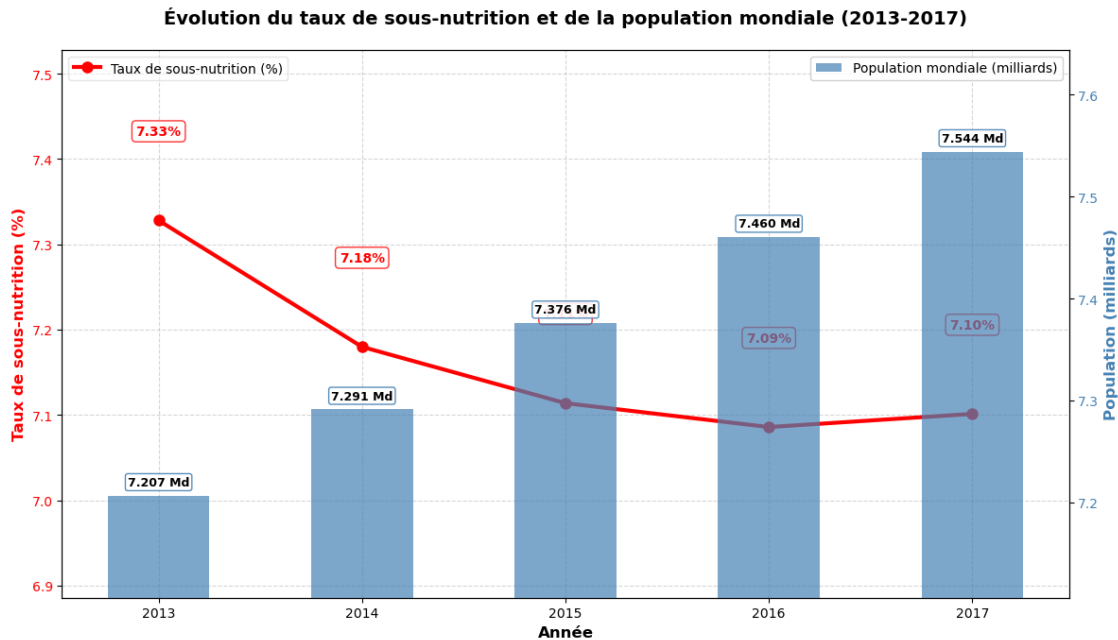
# --- Titre global ---
plt.title("Évolution du taux de sous-nutrition et de la population mondiale ↪
    ↪(2013-2017)",
    fontsize=14, fontweight='bold', pad=20)

# --- Légendes ---
ax1.legend(loc='upper left', framealpha=0.5)
ax2.legend(loc='upper right', framealpha=0.9)

plt.xticks([2013, 2014, 2015, 2016, 2017])
plt.tight_layout()
plt.show()

# Afficher les données pour comprendre les variations
print("\n" + "="*50)
print("ANALYSE DES DONNÉES :")
print("="*50)
for i, année in enumerate(taux_mondial["Année"]):
    print(f"{année}: Sous-nutrition = {taux_percent.iloc[i]:.2f}% | Population ↪
    ↪= {population_milliards.iloc[i]:.6f} Md")
print(f"\nVariation population: {pop_range:.6f} milliards")
print(f"Échelle population: {pop_min - 0.1:.3f} à {pop_max + 0.1:.3f} ↪
    ↪milliards")

```



=====

ANALYSE DES DONNÉES :

=====

2013: Sous-nutrition = 7.33% | Population = 7.206670 Md
 2014: Sous-nutrition = 7.18% | Population = 7.291347 Md
 2015: Sous-nutrition = 7.11% | Population = 7.375818 Md
 2016: Sous-nutrition = 7.09% | Population = 7.460004 Md
 2017: Sous-nutrition = 7.10% | Population = 7.543799 Md

Variation population: 0.337129 milliards
 Échelle population: 7.107 à 7.644 milliards

```
[60]: # Conversion en pourcentage
taux_percent = taux_mondial["Taux_sous_nutrition"] * 100

plt.figure(figsize=(9, 5))

# --- 1) Axe principal : taux de sous-nutrition (%)
ax1 = plt.gca()
ax1.plot(
    taux_mondial["Année"], taux_percent,
    marker='o', linewidth=2, label="Taux de sous-nutrition (%)"
)
ax1.set_xlabel("Année", fontsize=11)
ax1.set_ylabel("Taux de sous-nutrition (%)", fontsize=11)
```

```

ax1.tick_params(axis='y')
ax1.grid(True, linestyle='--', alpha=0.5)

# Ajouter les valeurs
for i, val in enumerate(taux_percent):
    ax1.text(
        taux_mondial["Année"].iloc[i],
        val + 0.05,
        f"{val:.2f} %",
        ha='center', fontsize=9
    )

# --- 2) Axe secondaire : population mondiale
ax2 = ax1.twinx()
ax2.plot(
    taux_mondial["Année"], taux_mondial["population_totale"],
    marker='s', linewidth=2, linestyle='--', color='grey',
    label="Population mondiale"
)
ax2.set_ylabel("Population totale", fontsize=11)
ax2.tick_params(axis='y')

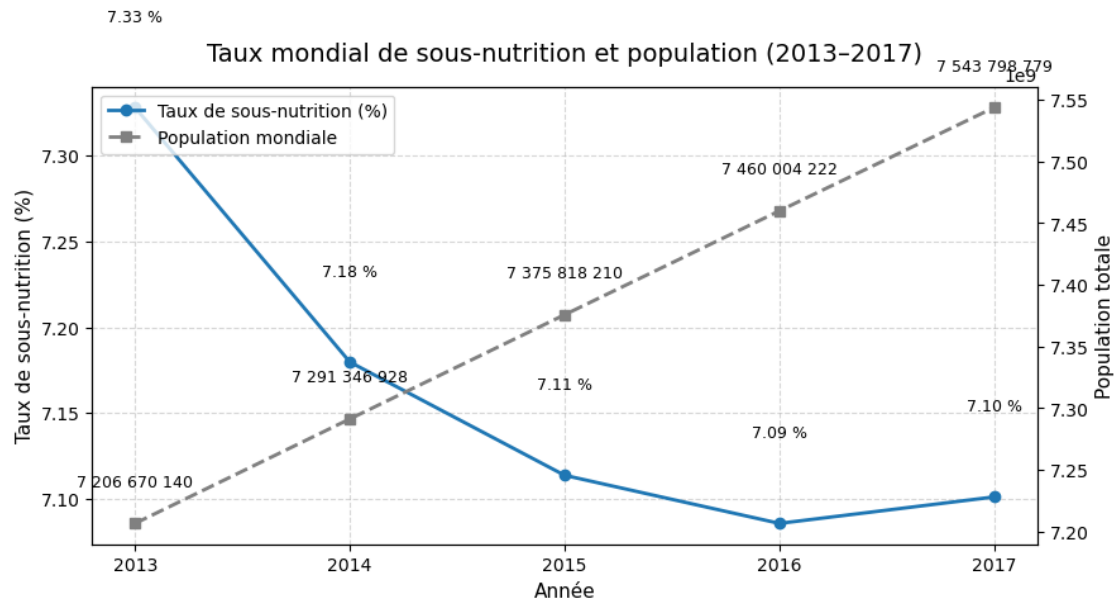
# Ajouter valeurs population
for i, val in enumerate(taux_mondial["population_totale"]):
    ax2.text(
        taux_mondial["Année"].iloc[i],
        val + 30_000_000,
        f"{val:,.0f}".replace(",", " "),
        ha='center', fontsize=9
    )

# --- Titre global ---
plt.title("Taux mondial de sous-nutrition et population (2013-2017)",
    ↪ fontsize=14, pad=15)

# --- Légendes combinées ---
lns1 = ax1.get_lines()
lns2 = ax2.get_lines()
plt.legend(lns1 + lns2, [l.get_label() for l in lns1 + lns2], loc="upper left")

plt.xticks([2013, 2014, 2015, 2016, 2017])
plt.tight_layout()
plt.show()

```



[61]: # Dictionnaire complet pays → continent (FAO compatible)

```
continent_mapping = {
    "Afghanistan": "Asie",
    "Afrique du Sud": "Afrique",
    "Albanie": "Europe",
    "Algérie": "Afrique",
    "Allemagne": "Europe",
    "Angola": "Afrique",
    "Arabie saoudite": "Asie",
    "Argentine": "Amérique du Sud",
    "Arménie": "Asie",
    "Australie": "Océanie",
    "Autriche": "Europe",
    "Azerbaïdjan": "Asie",
    "Bahamas": "Amérique du Nord",
    "Bangladesh": "Asie",
    "Belgique": "Europe",
    "Bénin": "Afrique",
    "Bhoutan": "Asie",
    "Biélorussie": "Europe",
    "Bolivie": "Amérique du Sud",
    "Botswana": "Afrique",
    "Brésil": "Amérique du Sud",
    "Bulgarie": "Europe",
    "Burkina Faso": "Afrique",
    "Burundi": "Afrique",
    "Cambodge": "Asie",
```

"Cameroun": "Afrique",
"Canada": "Amérique du Nord",
"Chili": "Amérique du Sud",
"Chine": "Asie",
"Chypre": "Europe",
"Colombie": "Amérique du Sud",
"Congo": "Afrique",
"Costa Rica": "Amérique du Nord",
"Côte d'Ivoire": "Afrique",
"Croatie": "Europe",
"Cuba": "Amérique du Nord",
"Danemark": "Europe",
"Djibouti": "Afrique",
"Dominique": "Amérique du Nord",
"Égypte": "Afrique",
"Émirats arabes unis": "Asie",
"Équateur": "Amérique du Sud",
"Érythrée": "Afrique",
"Espagne": "Europe",
"Estonie": "Europe",
"Eswatini": "Afrique",
"États-Unis": "Amérique du Nord",
"Éthiopie": "Afrique",
"Fidji": "Océanie",
"Finlande": "Europe",
"France": "Europe",
"Gabon": "Afrique",
"Gambie": "Afrique",
"Géorgie": "Asie",
"Ghana": "Afrique",
"Grèce": "Europe",
"Guatemala": "Amérique du Nord",
"Guinée": "Afrique",
"Guinée-Bissau": "Afrique",
"Guinée équatoriale": "Afrique",
"Haïti": "Amérique du Nord",
"Honduras": "Amérique du Nord",
"Hongrie": "Europe",
"Inde": "Asie",
"Indonésie": "Asie",
"Irak": "Asie",
"Iran": "Asie",
"Irlande": "Europe",
"Islande": "Europe",
"Israël": "Asie",
"Italie": "Europe",
"Jamaïque": "Amérique du Nord",

"Japon": "Asie",
"Jordanie": "Asie",
"Kazakhstan": "Asie",
"Kenya": "Afrique",
"Kirghizistan": "Asie",
"Kiribati": "Océanie",
"Koweït": "Asie",
"Laos": "Asie",
"Lesotho": "Afrique",
"Lettonie": "Europe",
"Liban": "Asie",
"Libéria": "Afrique",
"Libye": "Afrique",
"Lituanie": "Europe",
"Luxembourg": "Europe",
"Madagascar": "Afrique",
"Malaisie": "Asie",
"Malawi": "Afrique",
"Mali": "Afrique",
"Maroc": "Afrique",
"Mauritanie": "Afrique",
"Mexique": "Amérique du Nord",
"Moldavie": "Europe",
"Mongolie": "Asie",
"Mozambique": "Afrique",
"Namibie": "Afrique",
"Népal": "Asie",
"Niger": "Afrique",
"Nigeria": "Afrique",
"Norvège": "Europe",
"Nouvelle-Zélande": "Océanie",
"Oman": "Asie",
"Ouganda": "Afrique",
"Ouzbékistan": "Asie",
"Pakistan": "Asie",
"Papouasie-Nouvelle-Guinée": "Océanie",
"Pays-Bas": "Europe",
"Pérou": "Amérique du Sud",
"Philippines": "Asie",
"Pologne": "Europe",
"Portugal": "Europe",
"Qatar": "Asie",
"République centrafricaine": "Afrique",
"République démocratique du Congo": "Afrique",
"République dominicaine": "Amérique du Nord",
"République tchèque": "Europe",
"Roumanie": "Europe",

```

    "Royaume-Uni": "Europe",
    "Russie": "Europe",
    "Rwanda": "Afrique",
    "Sénégal": "Afrique",
    "Serbie": "Europe",
    "Sierra Leone": "Afrique",
    "Singapour": "Asie",
    "Slovaquie": "Europe",
    "Slovénie": "Europe",
    "Somalie": "Afrique",
    "Soudan": "Afrique",
    "Soudan du Sud": "Afrique",
    "Sri Lanka": "Asie",
    "Suède": "Europe",
    "Suisse": "Europe",
    "Suriname": "Amérique du Sud",
    "Syrie": "Asie",
    "Tadjikistan": "Asie",
    "Tanzanie": "Afrique",
    "Tchad": "Afrique",
    "Thaïlande": "Asie",
    "Togo": "Afrique",
    "Tunisie": "Afrique",
    "Turkménistan": "Asie",
    "Turquie": "Asie",
    "Ukraine": "Europe",
    "Uruguay": "Amérique du Sud",
    "Venezuela": "Amérique du Sud",
    "Vietnam": "Asie",
    "Yémen": "Asie",
    "Zambie": "Afrique",
    "Zimbabwe": "Afrique"
}

```

```

def trouver_continent(pays):
    return continent_mapping.get(pays, "Autres")

```

```

[62]: # Ajouter le continent à chaque pays
resultat["Continent"] = resultat["Zone"].apply(trouver_continent)

# Calcul du taux par pays
resultat["Taux_pays"] = (resultat["sous_nutrition"] / resultat["Population"]) * 100

import plotly.express as px

# Carte choroplèthe

```

```

fig = px.choropleth(
    resultat,
    locations="Zone",
    locationmode="country names",
    color="Taux_pays",
    hover_name="Zone",
    hover_data={"Taux_pays": ':.2f', "Population": True, "Continent": True},
    color_continuous_scale="YlOrRd",
    title="Taux de sous-nutrition par pays (2017)",
    width=950,
    height=550
)
fig.update_geos(showcoastlines=False, showframe=False)
fig.update_layout(coloraxis_colorbar=dict(title="Taux (%)"))
fig.show()

```

```

[63]: # Les 15 pays les plus touchés
classement = resultat.sort_values(by="Taux_pays", ascending=False).copy()
classement["Taux_pays"] = classement["Taux_pays"].round(2)

fig = px.bar(
    classement.head(15),
    x="Taux_pays",
    y="Zone",
    orientation='h',
    color="Taux_pays",
    color_continuous_scale="Reds",
    height=600,
    width=750,
    labels={"Taux_pays": "Taux de sous-nutrition (%)", "Zone": "Pays"},
    title="Les 15 pays les plus touchés par la sous-nutrition en 2017 (%)"
)
fig.update_layout(
    yaxis={'categoryorder': 'total ascending'}, # barre la plus haute en haut
    coloraxis_colorbar=dict(title="Taux (%)"),
    font=dict(size=11)
)
fig.show()

```

```

[64]: # 3.2 - Nombre théorique de personne qui pourrait être nourries

```

```

[65]: # Combien mange en moyenne un être humain ? Source => FAO appelle les besoins
      ↪ énergétiques alimentaires (DER), et pour mesurer la sous-alimentation ils
      ↪ utilisent un seuil autour de 1 800-2 000 kcal/personne/jour (ça varie un peu
      ↪ selon les pays, l'âge, l'activité physique, etc
DER = 2000 # Besoin énergétique minimum en kcal/jour

```

```
[66]: # D'abord vérifier que la sélection de colonnes marche :
test = population_2017[['Zone', 'Population']]
print(test.head())
```

```

      Zone  Population
4   Afghanistan  36296113.0
10  Afrique du Sud  57009756.0
16      Albanie   2884169.0
22      Algérie  41389189.0
28    Allemagne  82658409.0
```

```
[67]: # Filtrer la population pour 2017
# Joindre les deux tableaux : dispo alimentaire datant de 2017 et reprendre
# population_2017 (3.1)
# On assemble sur la colonne "Zone" (pays) inner pour garder seulement les
# pays communs

dispo_pop = pd.merge(dispo_alimentaire, population_2017[["Zone",
# "Population"]], on="Zone", how="inner")

# affichage du nouveau Dataframe
print("Aperçu du DataFrame fusionné appelé dispo_pop :")
display(dispo_pop.head())

# voir nombre de lignes et colonnes
print(dispo_pop.shape)
```

Aperçu du DataFrame fusionné appelé dispo_pop :

	Zone	Produit	Origine	Aliments pour animaux \
0	Afghanistan	Abats Comestible	animale	0.0
1	Afghanistan	Agrumes, Autres	vegetale	0.0
2	Afghanistan	Aliments pour enfants	vegetale	0.0
3	Afghanistan	Ananas	vegetale	0.0
4	Afghanistan	Bananes	vegetale	0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
0	0.0	5.0
1	0.0	1.0
2	0.0	1.0
3	0.0	0.0
4	0.0	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
0	1.72
1	1.29
2	0.06
3	0.00

4		2.70		
---	--	------	--	--

	Disponibilité de matière grasse en quantité (g/personne/jour) \	
0	0.20	
1	0.01	
2	0.01	
3	0.00	
4	0.02	

	Disponibilité de protéines en quantité (g/personne/jour) \	
0	0.77	
1	0.02	
2	0.03	
3	0.00	
4	0.05	

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53000000.0	0.0	0.0
1	41000000.0	2000000.0	40000000.0
2	2000000.0	0.0	2000000.0
3	0.0	0.0	0.0
4	82000000.0	0.0	82000000.0

	Nourriture	Pertes	Production	Semences	Traitement \
0	53000000.0	0.0	53000000.0	0.0	0.0
1	39000000.0	2000000.0	3000000.0	0.0	0.0
2	2000000.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	82000000.0	0.0	0.0	0.0	0.0

	Variation de stock	Population
0	0.0	36296113.0
1	0.0	36296113.0
2	0.0	36296113.0
3	0.0	36296113.0
4	0.0	36296113.0

(15416, 19)

```
[68]: # Création de la colonne dispo_kcal totale (kcal/personne/jour * population)
      ↪ dans dispo_pop
dispo_pop["Disponibilité_totale_kcal"] = (dispo_pop["Disponibilité alimentaire_
      ↪ (Kcal/personne/jour)"] * dispo_pop["Population"])

# Calcul du nombre de personnes nourries par produit pour chaque pays
dispo_pop["nb_personnes_theoriques_nourries"] =
      ↪ (dispo_pop["Disponibilité_totale_kcal"] / DER) # => valeur 2 000 (63)
```

```
dispo_pop.head()
```

```
[68]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale	0.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	
3	Afghanistan	Ananas	vegetale	0.0	
4	Afghanistan	Bananes	vegetale	0.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	0.0	5.0	
1	0.0	1.0	
2	0.0	1.0	
3	0.0	0.0	
4	0.0	4.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
0	0.20	
1	0.01	
2	0.01	
3	0.00	
4	0.02	

	Disponibilité de protéines en quantité (g/personne/jour)	\
0	0.77	
1	0.02	
2	0.03	
3	0.00	
4	0.05	

	Disponibilité intérieure	...	Importations - Quantité	Nourriture	\
0	53000000.0	...	0.0	53000000.0	
1	41000000.0	...	40000000.0	39000000.0	
2	2000000.0	...	2000000.0	2000000.0	
3	0.0	...	0.0	0.0	
4	82000000.0	...	82000000.0	82000000.0	

	Pertes	Production	Semences	Traitement	Variation de stock	\
0	0.0	53000000.0	0.0	0.0	0.0	
1	2000000.0	3000000.0	0.0	0.0	0.0	

2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

	Population	Disponibilité_totale_kcal	nb_personnes_theoriques_nourries
0	36296113.0	181480565.0	90740.2825
1	36296113.0	36296113.0	18148.0565
2	36296113.0	36296113.0	18148.0565
3	36296113.0	0.0	0.0000
4	36296113.0	145184452.0	72592.2260

[5 rows x 21 columns]

```
[69]: # calcul du Total de toute la nourriture disponible pour chaque pays

country_nutrition = (
    dispo_pop.groupby("Zone", as_index=False) # Groupe par pays
    .agg({
        "Disponibilité_totale_kcal": "sum",      # SOMME des kcal totales
        "nb_personnes_theoriques_nourries": "sum", # SOMME des personnes
        ↪nourries
        "Population": "first"                    # Prend la population
        ↪(même valeur pour le m^ pays)
    })
)
```

```
[70]: # calcul de la population total de 2017
pop_mondiale_2017 = country_nutrition["Population"].sum()

# calcul des Kcal dispo de tous les pays
kcal_totales_monde = country_nutrition["Disponibilité_totale_kcal"].sum()

# Calcul du nombre d'humains pouvant être nourris via nbre de calorie par jr:
↪Kcal dispo de tous les pays/2000 Kcal par jour
nb_personnes_theoriques_monde = kcal_totales_monde / DER

print(f"Population mondiale 2017 : {pop_mondiale_2017:,.0f}")
print(f"Disponibilité totale mondiale : {kcal_totales_monde:,.0f} kcal/jour")
print(f"Personnes pouvant être nourries : {nb_personnes_theoriques_monde:,.0f}")
```

Population mondiale 2017 : 7,291,900,830
Disponibilité totale mondiale : 20,918,984,627,331 kcal/jour
Personnes pouvant être nourries : 10,459,492,314

```
[71]: # Affichage des données via graphique histogrammes

fig = go.Figure()
```

```

# Barre pour la population réelle
fig.add_trace(go.Bar(
    x=["Population réelle"],
    y=[pop_mondiale_2017],
    name="Population réelle",
    marker_color="steelblue",
    text=f"{pop_mondiale_2017:,.0f}",
    textposition="outside"
))

# Barre pour la population nourrie théorique
fig.add_trace(go.Bar(
    x=["Population pouvant être nourrie"],
    y=[nb_personnes_theoriques_monde],
    name="Population théorique nourrie",
    marker_color="darkorange",
    text=f"{nb_personnes_theoriques_monde:,.0f}",
    textposition="outside"
))

# Mise en forme du graphique
fig.update_layout(
    title="Capacité alimentaire mondiale vs Population réelle (2017)",
    yaxis_title="Nombre de personnes",
    template="simple_white",
    width=750,
    height=500,
    showlegend=True
)

fig.show()

```

[72]: # 3.3 - Nombre théorique de personne qui pourrait être nourrie avec les
↳ produits végétaux

```

[73]: # Transfert des données avec les végétaux dans un nouveau dataframe
# Création d'un nouveau DataFrame avec uniquement les produits d'origine
↳ végétale
df_vegetaux = dispo_alimentaire[dispo_alimentaire['Origine'] == 'vegetale'].
↳ copy()

print(f"Nombre de lignes : {len(df_vegetaux)}")

# Affiche les premières lignes (par défaut 5)
df_vegetaux.head()

```

Nombre de lignes : 11896

[73]:

	Zone	Produit	Origine	Aliments pour animaux	\
1	Afghanistan	Agrumes, Autres	vegetale		0.0
2	Afghanistan	Aliments pour enfants	vegetale		0.0
3	Afghanistan	Ananas	vegetale		0.0
4	Afghanistan	Bananes	vegetale		0.0
6	Afghanistan	Bière	vegetale		0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
1	0.0		1.0
2	0.0		1.0
3	0.0		0.0
4	0.0		4.0
6	0.0		0.0

	Disponibilité alimentaire en quantité (kg/personne/an)	\
1	1.29	
2	0.06	
3	0.00	
4	2.70	
6	0.09	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
1	0.01	
2	0.01	
3	0.00	
4	0.02	
6	0.00	

	Disponibilité de protéines en quantité (g/personne/jour)	\
1	0.02	
2	0.03	
3	0.00	
4	0.05	
6	0.00	

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	\
1	41000000.0	2000000.0	40000000.0	
2	2000000.0	0.0	2000000.0	
3	0.0	0.0	0.0	
4	82000000.0	0.0	82000000.0	
6	3000000.0	0.0	3000000.0	

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
1	39000000.0	2000000.0	3000000.0	0.0	0.0	0.0
2	2000000.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	82000000.0	0.0	0.0	0.0	0.0	0.0

6	3000000.0	0.0	0.0	0.0	0.0	0.0
---	-----------	-----	-----	-----	-----	-----

```
[74]: # Joindre la population (2017) à df_vegetaux pour chaque pays (Zone)
df_veg_pop = pd.merge(df_vegetaux, population_2017[['Zone', 'Population']],
                      on='Zone', how='left')

# Affiche les premières lignes (par défaut 5)
df_veg_pop.head()
```

```
[74]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Agrumes, Autres	vegetale	0.0	
1	Afghanistan	Aliments pour enfants	vegetale	0.0	
2	Afghanistan	Ananas	vegetale	0.0	
3	Afghanistan	Bananes	vegetale	0.0	
4	Afghanistan	Bière	vegetale	0.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	0.0	1.0	
1	0.0	1.0	
2	0.0	0.0	
3	0.0	4.0	
4	0.0	0.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.29	
1	0.06	
2	0.00	
3	2.70	
4	0.09	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
0	0.01	
1	0.01	
2	0.00	
3	0.02	
4	0.00	

	Disponibilité de protéines en quantité (g/personne/jour)	\
0	0.02	
1	0.03	
2	0.00	
3	0.05	
4	0.00	

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	\
0	41000000.0	2000000.0	40000000.0	
1	2000000.0	0.0	2000000.0	

2	0.0	0.0	0.0
3	82000000.0	0.0	82000000.0
4	3000000.0	0.0	3000000.0

	Nourriture	Pertes	Production	Semences	Traitement \
0	39000000.0	2000000.0	3000000.0	0.0	0.0
1	2000000.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	82000000.0	0.0	0.0	0.0	0.0
4	3000000.0	0.0	0.0	0.0	0.0

	Variation de stock	Population
0	0.0	36296113.0
1	0.0	36296113.0
2	0.0	36296113.0
3	0.0	36296113.0
4	0.0	36296113.0

```
[75]: # Calculer la quantité totale de kcal disponibles par ligne (kcal/jour pour
      ↪ tout le pays, pour ce produit)
df_veg_pop['dispo_kcal_pays_produit'] = df_veg_pop['Disponibilité alimentaire
      ↪ (Kcal/personne/jour)'] * df_veg_pop['Population']
```

```
[76]: # Somme mondiale des kcal végétales (kcal/jour)
somme_kcal_vegetales_mondiale = df_veg_pop['dispo_kcal_pays_produit'].sum()
print("Kcal végétales totales disponibles (kcal/jour) :",
      ↪ somme_kcal_vegetales_mondiale)
```

Kcal végétales totales disponibles (kcal/jour) : 17260764211501.0

```
[77]: # Calcul du nombre d'humains pouvant être nourris avec les végétaux - Rappel
      ↪ DER 2000 l61

nb_personnes_nourries_veg = somme_kcal_vegetales_mondiale / DER
print(f"Nombre d'humains pouvant être nourris avec les végétaux :
      ↪ {nb_personnes_nourries_veg:,.0f} personnes")
```

Nombre d'humains pouvant être nourris avec les végétaux : 8,630,382,106 personnes

```
[78]: # Séparation des produits: végétal / animal
df_veg = dispo_alimentaire[dispo_alimentaire["Origine"] == "vegetale"].copy()
df_animal = dispo_alimentaire[dispo_alimentaire["Origine"] == "animale"].copy()

# Merge
df_veg = df_veg.merge(population_2017, on="Zone", how="left")
df_animal = df_animal.merge(population_2017, on="Zone", how="left")
```

```

# Kcal totale végétal / animal
df_veg["kcal_totales"] = df_veg["Disponibilité alimentaire (Kcal/personne/
↪jour)"] * df_veg["Population"]
df_animal["kcal_totales"] = df_animal["Disponibilité alimentaire (Kcal/personne/
↪jour)"] * df_animal["Population"]

kcal_veg = df_veg["kcal_totales"].sum()
kcal_animal = df_animal["kcal_totales"].sum()

print("Kcal végétales :", kcal_veg)
print("Kcal animales :", kcal_animal)

```

Kcal végétales : 17260764211501.0

Kcal animales : 3658220415830.0

```

[79]: # Données (déjà calculées)
kcal_vegetal = 17206764211501.0
kcal_animal = 3658220415830.0

labels = ["Végétal", "Animal"]
sizes = [kcal_vegetal, kcal_animal]

# Couleurs (FAO / turquoise)
couleur_vegetal = "#2ecc71" # vert FAO-style
couleur_animal = "#00AEB3" # turquoise FAO pour animal
colors = [couleur_vegetal, couleur_animal]

fig, ax = plt.subplots(figsize=(5, 5)) # <<< PLUS PETIT

# Donut chart compact
wedges, texts, autotexts = ax.pie(
    sizes,
    labels=labels,
    colors=colors,
    startangle=90,
    autopct="%.1f%%",
    pctdistance=0.75, # pourcentages plus près du centre
    textprops={"fontsize": 10, "color": "black"} # <<< police réduite
)

# Cercle blanc pour effet donut
centre_circle = plt.Circle((0, 0), 0.55, fc="white")
fig.gca().add_artist(centre_circle)

# Titre compact
ax.set_title(
    "Répartition mondiale des calories (2017)\nVégétal vs Produits animaux",

```

```

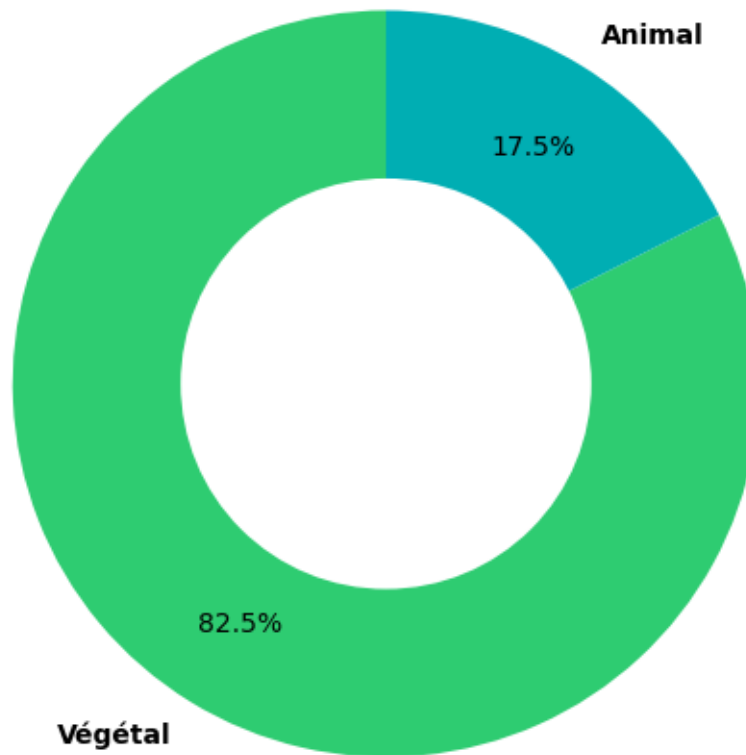
    fontsize=11,          # <<< titre plus petit
    fontweight="bold",
    pad=10                # <<< moins d'espace
)

# Labels en gras, version compacte
for t in texts:
    t.set_fontweight("bold")
    t.set_fontsize(10)

ax.axis("equal")
plt.tight_layout()
plt.show()

```

Répartition mondiale des calories (2017) Végétal vs Produits animaux



```

[80]: # Calories totales végétales par produit avec groupby
      kcal_par_produit = df_veg.groupby("Produit")["kcal_totales"].sum()

      # Trier par ordre décroissant

```

```

kcal_sorted = kcal_par_produit.sort_values(ascending=False)

# Afficher les 14 produits les + consommées parmi les végétaux
top14 = kcal_sorted.head(14)

# Regrouper les plus petits ensemble
autres = kcal_sorted.iloc[14:].sum()

# Faire la liste finale les 14 et les autres ensemble
kcal_final = top14.copy()
kcal_final["Autres produits végétaux"] = autres

# Conversion en %
kcal_final_pct = (kcal_final / kcal_par_produit.sum()) * 100

```

```

[81]: # Identification lignes produits => noms des produits; valeurs => % en vu du
      ↪ graphique
produits = kcal_final_pct.index.tolist()
valeurs = kcal_final_pct.values

```

```

[82]: # Création d'un dictionnaire Définition de la palette naturelle
palette = {
    "Riz": "#f5f5dc",          # Beige très clair (riz blanc)
    "Blé": "#daa520",          # Doré / Goldenrod
    "Sucre": "#c49e6c",         # Caramel / brun clair
    "Maïs": "#ffd700",          # Jaune vif (maïs)
    "Huile de Soja": "#cddc39", # Vert soja (vert jaunâtre pâle)
    "Légumes": "#4caf50",       # Vert moyen
    "Pommes de Terre": "#b5651d", # Terre cuite / brun rougeâtre
    "Huile de Palme": "#ff5722", # Rouge orangé
    "Manioc": "#f0e5d6",         # Beige rosé (manioc)
    "Légumineuses": "#8f9779",   # Vert kaki / brun-vert
    "Huile de Tournesol": "#ffb74d", # Orange doré (couleur tournesol)
    "Bière": "#d35400",          # Ambré rougeâtre (modifié pour éviter le
    ↪ doublon)
    "Huile de Colza&Moutarde": "#b4a647", # Jaune kaki (couleur moutarde)
    "Fruits": "#ff99cc",         # Rose pâle (fruits divers)
    "Autres": "#d3d3d3"          # Gris clair (catégorie fourre-tout)
}

```

```

[83]: # Fonction pour attribuer la bonne couleur selon le produit
def couleur_produit(produit):
    if "Riz" in produit:
        return palette["Riz"]
    elif "Blé" in produit:
        return palette["Blé"]
    elif "Sucre" in produit:

```

```

        return palette["Sucre"]
    elif "Maïs" in produit:
        return palette["Maïs"]
    elif "Huile de Soja" in produit:
        return palette["Huile de Soja"]
    elif "Légumes" in produit:
        return palette["Légumes"]
    elif "Pommes de Terre" in produit:
        return palette["Pommes de Terre"]
    elif "Huile de Palme" in produit:
        return palette["Huile de Palme"]
    elif "Manioc" in produit:
        return palette["Manioc"]
    elif "Légumineuses" in produit:
        return palette["Légumineuses"]
    elif "Huile de Tournesol" in produit:
        return palette["Huile de Tournesol"]
    elif "Bière" in produit:
        return palette["Bière"]
    elif "Huile de Colza" in produit or "Moutarde" in produit:
        return palette["Huile de Colza&Moutarde"]
    elif "Fruits" in produit:
        return palette["Fruits"]
    else:
        return palette["Autres"] # catégorie fourre-tout

```

```

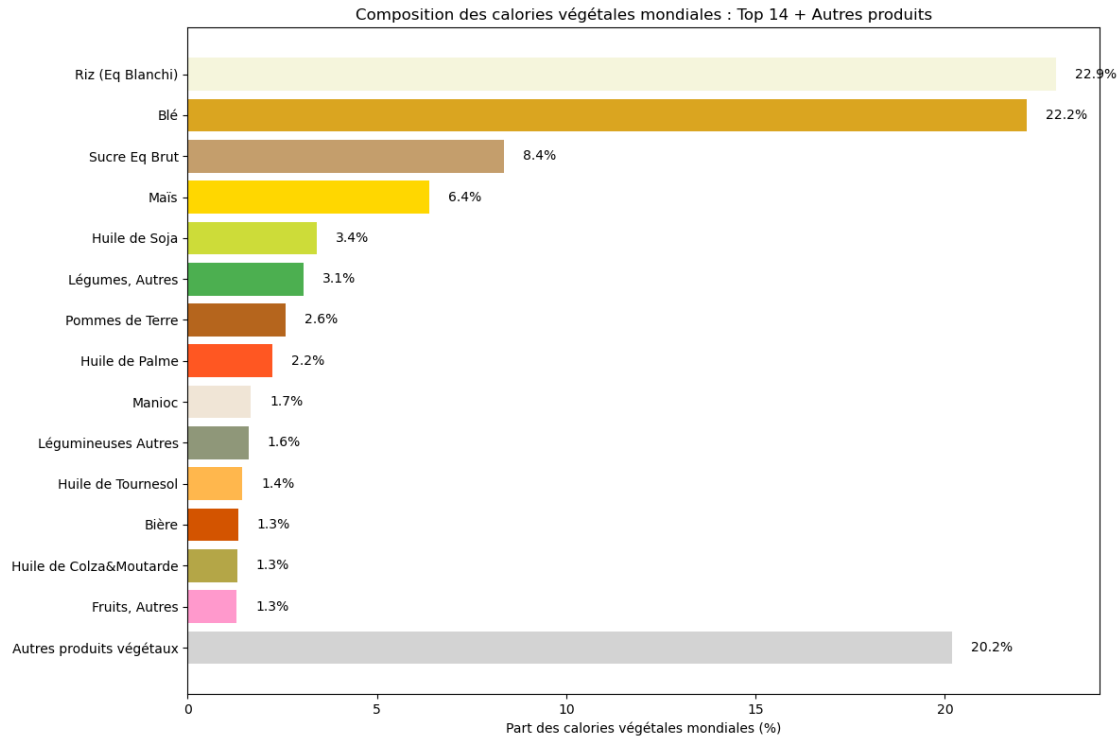
[84]: # Application de la palette
couleurs = [couleur_produit(p) for p in produits]

# Graphique final
plt.figure(figsize=(12, 8))
plt.barh(produits, valeurs, color=couleurs)
plt.gca().invert_yaxis()
plt.xlabel("Part des calories végétales mondiales (%)")
plt.title("Composition des calories végétales mondiales : Top 14 + Autres_
↳produits")

# Ajout des pourcentages
for i, v in enumerate(valeurs):
    plt.text(v + 0.5, i, f"{v:.1f}%", va='center')

plt.tight_layout()
plt.show()

```



```
[85]: # Calories totales animales par produit avec groupby
kcal_par_produit_animal = df_animal.groupby("Produit")["kcal_totales"].sum()

# Trier par ordre décroissant
kcal_sorted_animal = kcal_par_produit_animal.sort_values(ascending=False)

# Afficher les produits les plus consommés / animaux
top10_animal = kcal_sorted_animal.head(10)

# Regrouper les plus petits ensemble
autres_animal = kcal_sorted_animal.iloc[10:].sum()

# Faire la liste finale les 10 et les autres ensemble
kcal_final_animal = top10_animal.copy()
kcal_final_animal["Autres produits animaux"] = autres_animal

# Conversion en %
kcal_final_pct_animal = (kcal_final_animal / kcal_par_produit_animal.sum()) * 100

[86]: # Identification lignes produits => noms des produits; valeurs => % en vu du graphique
produits_an = kcal_final_pct_animal.index.tolist()
```



```
valeurs_an = kcal_final_pct_animal.values
```

```
[87]: # Création d'un dictionnaire Définition de la palette naturelle
```

```
palette_animale = {  
    "Lait - Excl Beurre": "#A7C7E7",  
    "Viande de Suides": "#E09BA0",  
    "Viande de Volailles": "#F4D06F",  
    "Viande de Bovins": "#8A0303",  
    "Œufs": "#F7C500",  
    "Beurre, Ghee": "#F2DA63",  
    "Graisses Animales Crues": "#C4A484",  
    "Poissons Eau Douce": "#1E3A5F",  
    "Viande d'Ovins/Caprins": "#7A1F3D",  
    "Poissons Pelagiques": "#003f5c",  
    "Autres produits animaux": "#BFBFBF"  
}
```

```
[88]: # Fonction pour attribuer la bonne couleur selon le produit
```

```
def couleur_produit_animal(produit):  
  
    if "Lait - Excl Beurre" in produit:  
        return palette_animale["Lait - Excl Beurre"]  
  
    elif "Viande de Suides" in produit:  
        return palette_animale["Viande de Suides"]  
  
    elif "Viande de Volailles" in produit:  
        return palette_animale["Viande de Volailles"]  
  
    elif "Viande de Bovins" in produit:  
        return palette_animale["Viande de Bovins"]  
  
    elif "Œufs" in produit or "Oeufs" in produit:  
        return palette_animale["Œufs"]  
  
    elif "Beurre" in produit:  
        return palette_animale["Beurre, Ghee"]  
  
    elif "Graisses Animales Crues" in produit:  
        return palette_animale["Graisses Animales Crues"]  
  
    elif "Poissons Eau Douce" in produit:  
        return palette_animale["Poissons Eau Douce"]  
  
    elif "Viande d'Ovins" in produit or "Viande d'Ovins" in produit or  
↳ "Caprins" in produit:
```

```

    return palette_animale["Viande d'Ovins/Caprins"]

elif "Poissons Pelagiques" in produit:
    return palette_animale["Poissons Pelagiques"]

else:
    return palette_animale["Autres produits animaux"]

```

```

[89]: # Application de la palette animale
produits_an = kcal_final_pct_animal.index.tolist()
valeurs_an = kcal_final_pct_animal.values

couleurs_an = [couleur_produit_animal(p) for p in produits_an]

```

```

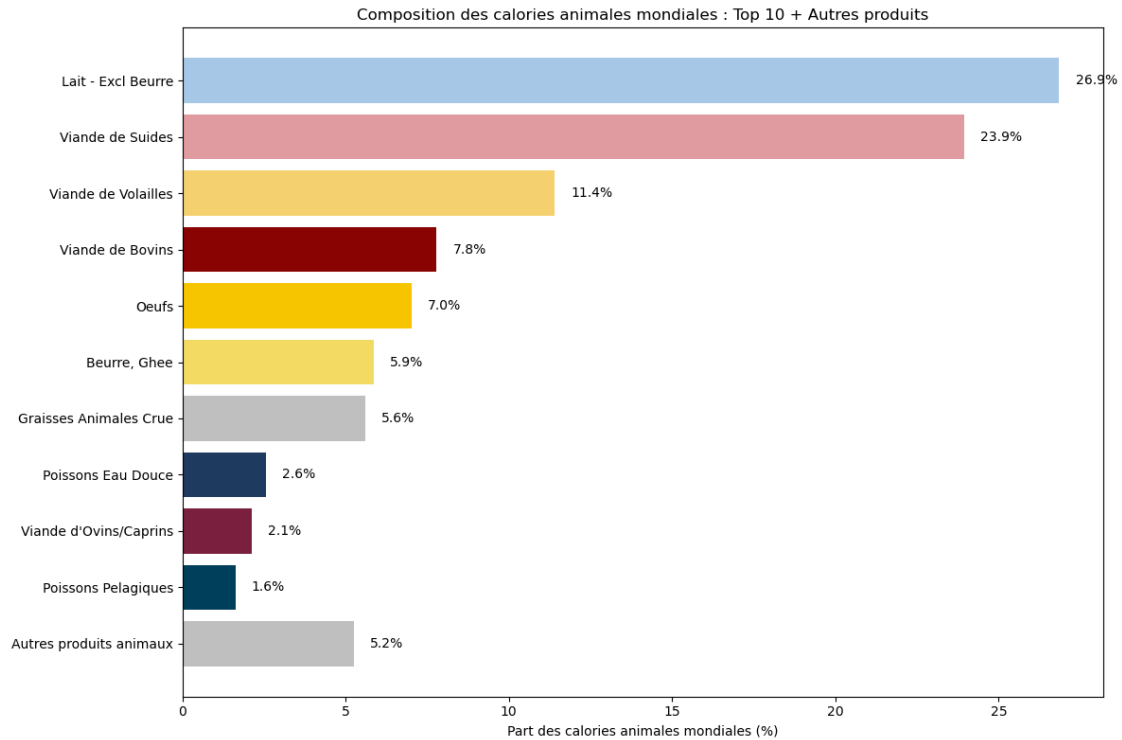
[90]: # Graphique
plt.figure(figsize=(12, 8))
plt.barh(produits_an, valeurs_an, color=couleurs_an)
plt.gca().invert_yaxis()

plt.xlabel("Part des calories animales mondiales (%)")
plt.title("Composition des calories animales mondiales : Top 10 + Autres_
↳ produits")

# Ajout des pourcentages
for i, v in enumerate(valeurs_an):
    plt.text(v + 0.5, i, f"{v:.1f}%", va='center')

plt.tight_layout()
plt.show()

```



```
[91]: # 3.4 - Utilisation de la disponibilité intérieure = COLONNE J
# Pour info: Disponibilité intérieure total
# ↳ Production+Importations-Exportations+Variation de stock
# Calcul de la disponibilité intérieure mondiale
dispo_totale = dispo_alimentaire['Disponibilité intérieure'].sum()
print(f"Disponibilité intérieure totale mondiale : {dispo_totale:,.0f} Kg")
```

Disponibilité intérieure totale mondiale : 9,848,994,000,000 Kg

```
[92]: # création d'une boucle for pour afficher les différentes valeurs en fonction
# ↳ des colonnes aliments pour animaux, pertes, nourritures,
# Étape 1 : Liste des colonnes à analyser - COLONNE D, N ,M = Disponibilité
# ↳ Alimentaire
colonnes_a_afficher = ["Aliments pour animaux", "Pertes", "Nourriture"]

# Boucle sur chaque colonne Avec for et calcul du total mondial pour cette
# ↳ colonne
for col in colonnes_a_afficher:
    total = dispo_alimentaire[col].sum()

# Affichage du résultat avec formatage
print(f" Total {col} : {total:,.0f}")
```

Total Aliments pour animaux : 1,304,245,000,000

Total Pertes : 453,698,000,000
Total Nourriture : 4,876,258,000,000

```
[93]: # Calcul des totaux avec disponibilité intérieure
colonnes_etendues = colonnes_a_afficher + ['Disponibilité intérieure']
totaux = {
    "Catégorie": colonnes_etendues,
    "Total mondial (Kg)": [dispo_alimentaire[col].sum() for col in
        ↪ colonnes_etendues]
}

# Création du DataFrame
tableau = pd.DataFrame(totaux)

# Formatage des nombres
tableau["Total mondial (Kg)"] = tableau["Total mondial (Kg)"].map(lambda x:
    ↪ f"{x:,.0f}")

# --- STYLE DESIGN COMPATIBLE ---
def style_table(df):
    styles = [
        {"selector": "th", "props": "background-color:#1e1e1e; color:white;
        ↪ padding:12px; font-weight:bold;"},
        {"selector": "td", "props": "padding:10px; font-size:14px;"},
        {"selector": "table", "props": "border-collapse:collapse; width:100%;
        ↪ border:2px solid #ddd; font-family:Segoe UI, sans-serif;"},
        {"selector": "tbody tr:nth-child(odd)", "props": "background-color:
        ↪ #f8f9fa;"},
        {"selector": "tbody tr:nth-child(even)", "props": "background-color:
        ↪ #ffffff;"},
        {"selector": "tbody tr:hover", "props": "background-color:#e3f2fd;
        ↪ transform:scale(1.01);"},
        {"selector": "tbody tr:last-child", "props": "background-color:#e8f5e8;
        ↪ font-weight:bold; border-top:2px solid #2e7d32;"}
    ]
    return df.style.set_table_styles(styles).set_properties(**{
        "text-align": "center",
        "border": "1px solid #bbb"
    }).set_caption(" Totaux mondiaux par catégorie - Vue d'ensemble")

# Application du style
styled_tableau = style_table(tableau)
styled_tableau
```

[93]: <pandas.io.formats.style.Styler at 0x16db94190>

```
[94]: # Rappel de la disponibilité

print("DESTINATION DE LA NOURRITURE MONDIALE :")
print(f"Total disponible : {dispo_totale:,.0f} Kg")

# Calcul des pourcentages
nourriture_pct = (dispo_alimentaire['Nourriture'].sum() / dispo_totale) * 100
animaux_pct = (dispo_alimentaire['Aliments pour animaux'].sum() / dispo_totale)
↳* 100
pertes_pct = (dispo_alimentaire['Pertes'].sum() / dispo_totale) * 100
autres_pct = 100 - nourriture_pct - animaux_pct - pertes_pct # pour obtenir un
↳résultat cohérente de la disponibilité alimentaire via dispo inte

# Affichage des réponses en %
print(f"Nourriture humaine : {nourriture_pct:.2f}%")
print(f"Aliments pour animaux : {animaux_pct:.2f}%")
print(f"Pertes : {pertes_pct:.2f}%")
print(f"Autres utilisations : {autres_pct:.2f}%")
```

```
DESTINATION DE LA NOURRITURE MONDIALE :
Total disponible : 9,848,994,000,000 Kg
Nourriture humaine : 49.51%
Aliments pour animaux : 13.24%
Pertes : 4.61%
Autres utilisations : 32.64%
```

```
[95]: # création du dictionnaire parts_pct => définir nom+%
parts_pct = {
    "Alimentation humaine": nourriture_pct,
    "Alimentation animale": animaux_pct,
    "Pertes": pertes_pct,
    "Autres usages": autres_pct
}
# définitions des étiquettes: labels=nom, values=valeurs
labels = list(parts_pct.keys())
values = list(parts_pct.values())

# Création d'une liste pour définir couleur pour chaque barre
colors = ["#2ecc71", # vert nourriture
          "#3498db", # bleu animaux
          "#e74c3c", # rouge pertes
          "#95a5a6"] # gris autres utilisations

# Graphique en barres horizontales
plt.figure(figsize=(8, 5))

bars = plt.barh(labels, values, color=colors)
```

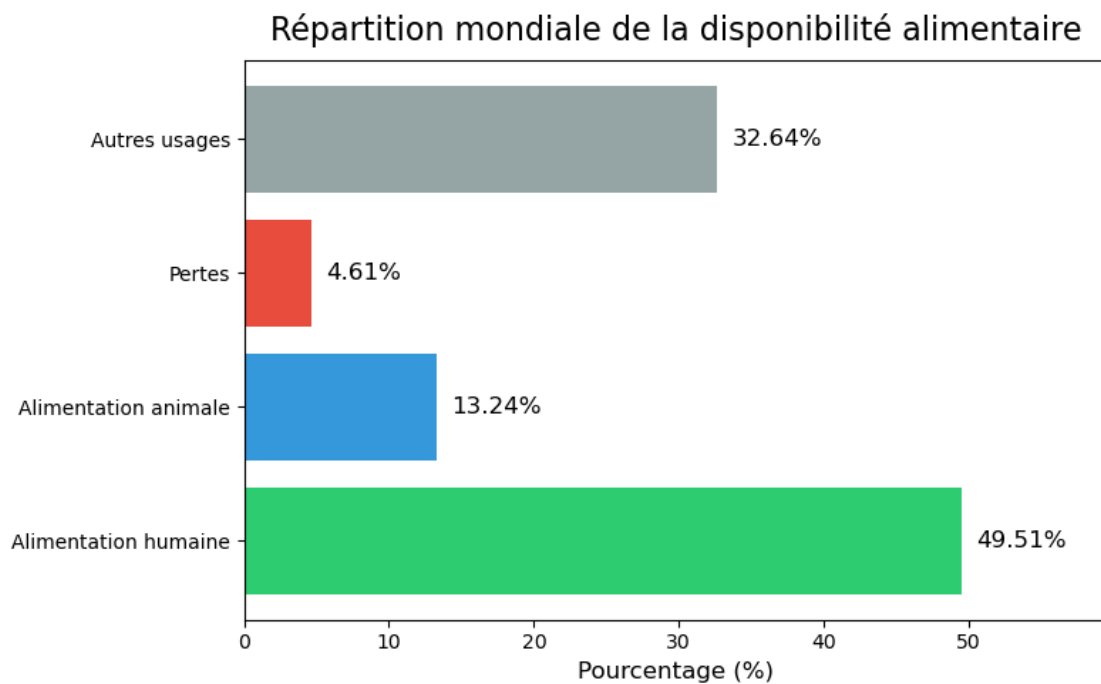
```

# Ajout des étiquettes de pourcentage à droite de chaque barre
for bar, value in zip(bars, values):
    plt.text(
        bar.get_width() + 1,                                # position X (un peu à
        ↪ droite de la barre)
        bar.get_y() + bar.get_height() / 2,                # position Y au centre
        ↪ vertical de la barre
        f"{value:.2f}%",                                     # texte
        va='center', fontsize=12
    )

plt.xlabel("Pourcentage (%)", fontsize=12)
plt.title("Répartition mondiale de la disponibilité alimentaire", fontsize=16,
    ↪ pad=10)
plt.xlim(0, max(values) + 10) # laisse de la place pour les étiquettes

plt.tight_layout()
plt.show()

```



[96]: # 3.5 - Utilisation des céréales¶

[97]: # Création d'une liste avec toutes les variables = produits considérés comme
 ↪ des céréales

```
liste_cereales = [
    "Blé",
    "Riz (Eq Blanchi)",
    "Maïs",
    "Orge",
    "Avoine",
    "Seigle",
    "Sorgho",
    "Mil",
    "Autres céréales"
]
```

```
[98]: # Création d'un dataframe avec les informations uniquement pour ces céréales#
      ↪Création d'un DataFrame contenant uniquement les céréales
df_cereales = dispo_alimentaire[dispo_alimentaire["Produit"].
      ↪isin(liste_cereales)].copy()

# Afficher le Dataframe
display(df_cereales.head())

print(f"Nombre de lignes : {len(df_cereales)}")
```

	Zone	Produit	Origine	Aliments pour animaux \
7	Afghanistan	Blé	vegetale	0.0
32	Afghanistan	Maïs	vegetale	200000000.0
40	Afghanistan	Orge	vegetale	360000000.0
47	Afghanistan	Riz (Eq Blanchi)	vegetale	0.0
67	Afrique du Sud	Avoine	vegetale	8000000.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
7	0.0	1369.0
32	0.0	21.0
40	0.0	26.0
47	0.0	141.0
67	0.0	5.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
7	160.23
32	2.50
40	2.92
47	13.82
67	0.75

	Disponibilité de matière grasse en quantité (g/personne/jour) \
7	4.69
32	0.30
40	0.24
47	0.27

67		0.09
----	--	------

	Disponibilité de protéines en quantité (g/personne/jour) \
7	36.91
32	0.56
40	0.79
47	2.70
67	0.16

	Disponibilité intérieure	Exportations - Quantité \
7	5.992000e+09	0.0
32	3.130000e+08	0.0
40	5.240000e+08	0.0
47	4.610000e+08	0.0
67	4.900000e+07	4000000.0

	Importations - Quantité	Nourriture	Pertes	Production \
7	1.173000e+09	4.895000e+09	775000000.0	5.169000e+09
32	1.000000e+06	7.600000e+07	31000000.0	3.120000e+08
40	1.000000e+07	8.900000e+07	52000000.0	5.140000e+08
47	1.190000e+08	4.220000e+08	24000000.0	3.420000e+08
67	2.500000e+07	4.000000e+07	1000000.0	2.800000e+07

	Semences	Traitement	Variation de stock
7	322000000.0	0.0	-350000000.0
32	5000000.0	0.0	0.0
40	22000000.0	0.0	0.0
47	14000000.0	0.0	0.0
67	1000000.0	0.0	0.0

Nombre de lignes : 1169

```
[99]: # Affichage de la proportion de céréales allant à l'alimentation animale

# Création de la colonne
df_cereales["Proportion animale"] = (df_cereales["Aliments pour animaux"] /
↳ df_cereales["Disponibilité intérieure"])

total_animaux = df_cereales["Aliments pour animaux"].sum()
total_dispo = df_cereales["Disponibilité intérieure"].sum()

proportion_globale = total_animaux / total_dispo
print(f"Proportion mondiale d'alimentation animale : {proportion_globale:.2%}")
```

Proportion mondiale d'alimentation animale : 36.23%

```
[100]: # Afficher le Dataframe
display(df_cereales.head())
```


	Zone	Produit	Origine	Aliments pour animaux \
7	Afghanistan	Blé	vegetale	0.0
32	Afghanistan	Maïs	vegetale	200000000.0
40	Afghanistan	Orge	vegetale	360000000.0
47	Afghanistan	Riz (Eq Blanchi)	vegetale	0.0
67	Afrique du Sud	Avoine	vegetale	8000000.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
7	0.0	1369.0
32	0.0	21.0
40	0.0	26.0
47	0.0	141.0
67	0.0	5.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
7	160.23
32	2.50
40	2.92
47	13.82
67	0.75

	Disponibilité de matière grasse en quantité (g/personne/jour) \
7	4.69
32	0.30
40	0.24
47	0.27
67	0.09

	Disponibilité de protéines en quantité (g/personne/jour) \
7	36.91
32	0.56
40	0.79
47	2.70
67	0.16

	Disponibilité intérieure	Exportations - Quantité \
7	5.992000e+09	0.0
32	3.130000e+08	0.0
40	5.240000e+08	0.0
47	4.610000e+08	0.0
67	4.900000e+07	4000000.0

	Importations - Quantité	Nourriture	Pertes	Production \
7	1.173000e+09	4.895000e+09	775000000.0	5.169000e+09
32	1.000000e+06	7.600000e+07	31000000.0	3.120000e+08
40	1.000000e+07	8.900000e+07	52000000.0	5.140000e+08
47	1.190000e+08	4.220000e+08	24000000.0	3.420000e+08
67	2.500000e+07	4.000000e+07	1000000.0	2.800000e+07

	Semences	Traitement	Variation de stock	Proportion_animale
7	322000000.0	0.0	-350000000.0	0.000000
32	5000000.0	0.0	0.0	0.638978
40	22000000.0	0.0	0.0	0.687023
47	14000000.0	0.0	0.0	0.000000
67	1000000.0	0.0	0.0	0.163265

[101]: *# Calcul par catégories (en kg) de comment les céréales sont utilisées*

```
total_nourriture = df_cereales["Nourriture"].sum()
total_animaux = df_cereales["Aliments pour animaux"].sum()
total_autres = (
    df_cereales["Semences"].sum() +
    df_cereales["Pertes"].sum() +
    df_cereales["Traitement"].sum() +
    df_cereales["Autres Utilisations"].sum()# on regroupe
    ↪semences+pertes+traitement+autres utilisations
)

# Addition de toutes les catégories du Total céréales utilisées
total_cereales = total_nourriture + total_animaux + total_autres

# avec fonction calcul du % de chaque catégorie
def pct(x):
    return 100 * x / total_cereales
```

[102]: *# Création graphique Sankey*

```
fig = go.Figure(data=[go.Sankey(
    node=dict(
        pad=25,
        thickness=25,
        line=dict(color="black", width=1),
        label=[
            "Céréales (totales)",
            "Alimentation humaine",
            "Alimentation animale",
            "Autres usages"
        ],
        color=["#A2D45E", "#4DA3FF", "#FF9A00", "#A0A0A0"]
    ),
    link=dict(
        source=[0, 0, 0],
        target=[1, 2, 3],
        value=[pct(total_nourriture), pct(total_animaux), pct(total_autres)],
        color=["#4DA3FF", "#FF9A00", "#A0A0A0"]
    )
)])
```

```

# ÉTIQUETTES NOIRES (ULTRA LISIBLES)
annotations = [
    dict(
        x=0.52, y=0.80,                                # Humaine
        text=f"{pct(total_nourriture):.2f}%",
        showarrow=False,
        font=dict(size=18, color="black", family="Arial")
    ),
    dict(
        x=0.52, y=0.45,                                # Animale
        text=f"{pct(total_animaux):.2f}%",
        showarrow=False,
        font=dict(size=18, color="black", family="Arial")
    ),
    dict(
        x=0.52, y=0.10,                                # Autres usages
        text=f"{pct(total_autres):.2f}%",
        showarrow=False,
        font=dict(size=18, color="black", family="Arial")
    )
]

fig.update_layout(
    title_text="Répartition mondiale de l'utilisation des céréales (uniquement,
↪ en %)",
    font_size=14,
    annotations=annotations,
    width=1100,
    height=450
)

fig.show()

```

```

[103]: # contrôle des résultats
pct_humains = pct(total_nourriture)
pct_animaux = pct(total_animaux)
pct_autres = pct(total_autres)

print(f"Humains : {pct_humains:.1f}%")
print(f"Animaux : {pct_animaux:.1f}%")
print(f"Autres : {pct_autres:.1f}%")

```

```

Humains : 42.6%
Animaux : 36.2%
Autres : 21.2%

```

```
[104]: # Regroupement par pays des catégories
df_pays = df_cereales.groupby("Zone").agg(
    total_animaux=("Aliments pour animaux", "sum"),
    total_humain=("Nourriture", "sum"),
    total_dispo=("Disponibilité intérieure", "sum")
)

# Calcul des proportions par pays
df_pays["Part_animale"] = df_pays["total_animaux"] / df_pays["total_dispo"]
df_pays["Part_humaine"] = df_pays["total_humain"] / df_pays["total_dispo"]

[105]: # Classement des pays en ordre décroissant = pays favorise les animeaux pour
↳ les alimenter avec céréales
top_animaux = df_pays.sort_values("Part_animale", ascending=False).head(10)
print("TOP 10 - Part la plus élevée d'utilisation animale :")
display(top_animaux[["Part_animale"]])
```

TOP 10 - Part la plus élevée d'utilisation animale :

Zone	Part_animale
Danemark	0.821289
Irlande	0.765290
Espagne	0.741387
Canada	0.726825
Arabie saoudite	0.708101
Croatie	0.702174
Serbie	0.699555
Chypre	0.688889
Pays-Bas	0.685453
Bélarus	0.659170

```
[106]: plt.figure(figsize=(10,6))

bars = plt.barh(
    top_animaux.index,
    top_animaux["Part_animale"],
    color="orange"
)

plt.title("Top 10 pays - Part des céréales destinées aux animaux (%)")
plt.xlabel("Part (%)")
plt.gca().invert_yaxis() # Pour avoir le TOP en haut

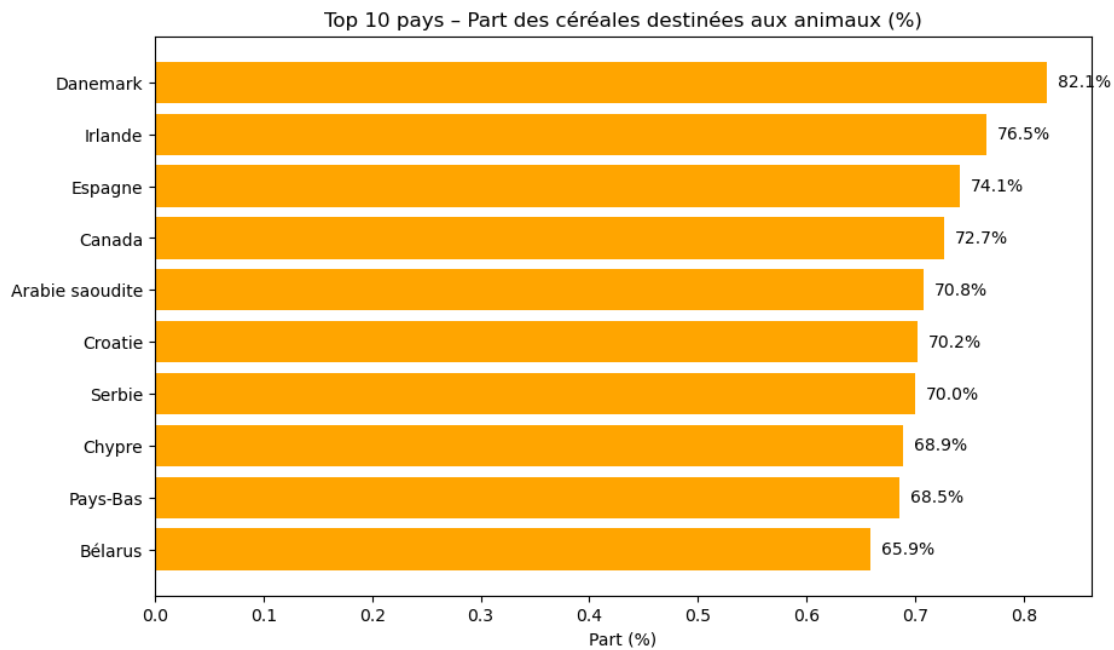
# Ajout des étiquettes en %
for bar in bars:
    width = bar.get_width()
    plt.text(
```

```

width + 0.01,          # position X légèrement à droite
bar.get_y() + bar.get_height()/2,
f"{width*100:.1f}%",    # conversion en %
va='center',
fontsize=10
)

plt.show()

```



```

[107]: # Classement des pays en ordre décroissant = pays favorise les humains pour
        ↪ les alimenter avec céréales
top_humain = df_pays.sort_values("Part_humaine", ascending=False).head(10)
print("TOP 10 - Part la plus élevée d'utilisation humaine :")
display(top_humain[["Part_humaine"]])

```

TOP 10 - Part la plus élevée d'utilisation humaine :

Zone	Part_humaine
Sao Tomé-et-Principe	1.000000
Chine - RAS de Hong-Kong	0.950318
Djibouti	0.941935
Timor-Leste	0.934010
Mauritanie	0.923505
Kiribati	0.923077
Lesotho	0.912109

Yémen	0.906588
Kenya	0.904349
Sénégal	0.903678

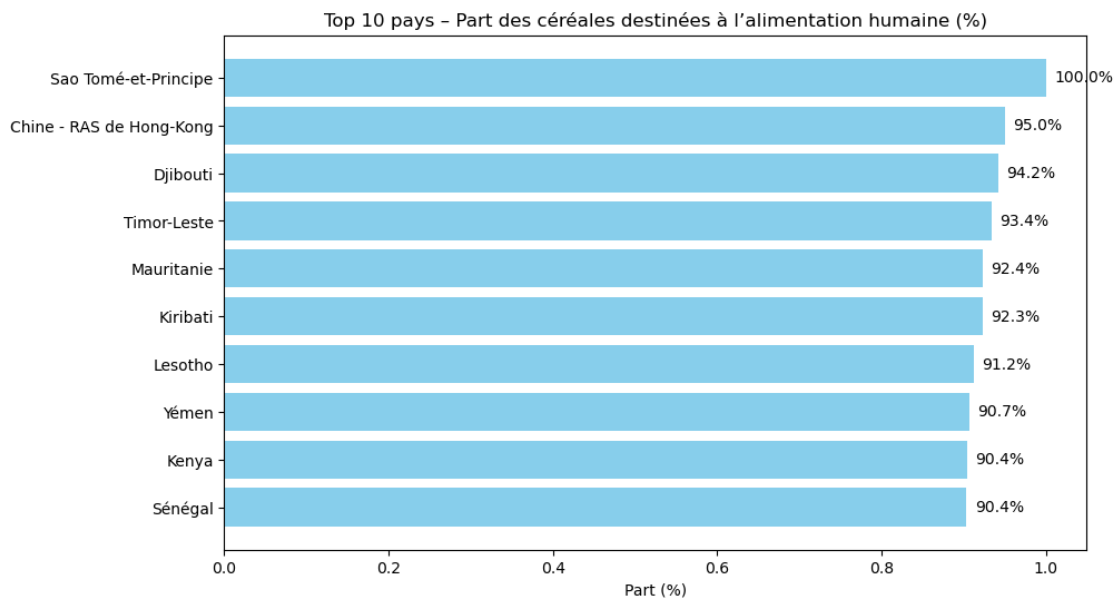
```
[108]: plt.figure(figsize=(10,6))

bars = plt.barh(
    top_humain.index,
    top_humain["Part_humaine"],
    color="skyblue"
)

plt.title("Top 10 pays - Part des céréales destinées à l'alimentation humaine_↵(%)")
plt.xlabel("Part (%)")
plt.gca().invert_yaxis()

# Ajout des étiquettes %
for bar in bars:
    width = bar.get_width()
    plt.text(
        width + 0.01,
        bar.get_y() + bar.get_height()/2,
        f"{width*100:.1f}%",
        va='center',
        fontsize=10
    )

plt.show()
```



```
[109]: # 3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en
↳ 2017
```

```
[110]: # Création de la colonne proportion par pays

resultat["proportion"] = (resultat["sous_nutrition"] / resultat["Population"])*
↳ 100

print(resultat.head())
```

	Zone	Année_x	Population	Année_y	sous_nutrition	Continent \
0	Afghanistan	2017	36296113.0	2016-2018	10500000.0	Asie
1	Afrique du Sud	2017	57009756.0	2016-2018	3100000.0	Afrique
2	Albanie	2017	2884169.0	2016-2018	100000.0	Europe
3	Algérie	2017	41389189.0	2016-2018	1300000.0	Afrique
4	Allemagne	2017	82658409.0	2016-2018	0.0	Europe

	Taux_pays	proportion
0	28.928718	28.928718
1	5.437666	5.437666
2	3.467203	3.467203
3	3.140917	3.140917
4	0.000000	0.000000

```
[111]: # Trier les pays par proportion décroissante
resultat = resultat.sort_values(by="proportion", ascending=False)
```

```
[112]: # affichage après trie des 10 pires pays
print(resultat[["Zone", "proportion"]].head(10))
```

	Zone	proportion
87	Haïti	48.259182
181	République populaire démocratique de Corée	47.188685
128	Madagascar	41.062924
122	Libéria	38.279742
119	Lesotho	38.249438
216	Tchad	37.957606
186	Rwanda	35.055619
145	Mozambique	32.810898
219	Timor-Leste	32.173531
0	Afghanistan	28.928718

```
[113]: # Faire graphique
# Sélection des 10 pays les plus touchés pour le graphique
top10 = resultat[["Zone", "proportion"]].head(10)
```

```

# Configuration de la taille du graphique
plt.figure(figsize=(6,6)) # largeur=10, hauteur=6

# Création du graphique en barres horizontales
plt.barh(
    top10["Zone"],          # noms des pays sur l'axe Y
    top10["proportion"],    # proportions sur l'axe X
    color='#0073E6'        # couleur bleu
)

# Titre et étiquettes des axes
plt.title("Pays avec la plus forte proportion de personnes sous-alimentés_↵
↵(2017)", fontsize=14, fontweight='bold')
plt.xlabel("Proportion de personnes sous-alimentées (%)", fontsize=12)
plt.ylabel("Pays", fontsize=12)

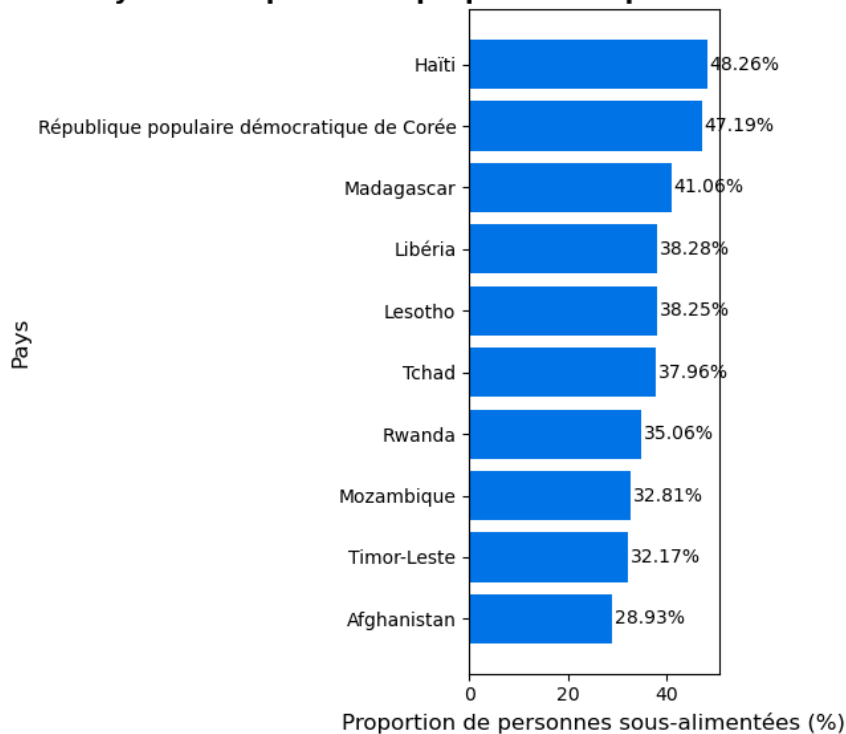
# Inverser l'ordre pour que le pays le plus touché soit en haut
plt.gca().invert_yaxis()

# Ajouter les valeurs numériques au bout de chaque barre
for i, value in enumerate(top10["proportion"]):
    plt.text(value + 0.5, i, f"{value:.2f}%", va='center', fontsize=10)

# Affichage final
plt.tight_layout() # ajuste l'espace pour éviter les chevauchements
plt.show()

```


Pays avec la plus forte proportion de personnes sous-alimentés (2017)



```
[114]: # Pour le table design: on selectionne table d'origine
top10 = resultat[["Zone", "proportion"]].head(10).copy()

# on arrondi les chiffres
top10["proportion"] = top10["proportion"].round(2)

# On renomme les colonnes pour un affichage propre
top10 = top10.rename(columns={
    "Zone": "Pays",
    "proportion": "Proportion de population sous-alimentée (%)"
})

# Stylisation du tableau ---
def highlight_max(s):
    """Met en évidence la valeur max."""
    is_max = s == s.max()
    return ['background-color: #ffb3b3' if v else '' for v in is_max]

styled_table = (
    top10.style
        .set_caption(" Top 10 des pays avec la plus forte sous-alimentation_
↪(2017)")
```

```

        .apply(highlight_max, subset=["Proportion de population sous-alimentée_
↪(%)"])
        .format({"Proportion de population sous-alimentée (%)": "{:.2f} %"})
        .set_properties(**{
            'text-align': 'center',
            'font-size': '14px'
        })
        .set_table_styles([
            {
                'selector': 'th',
                'props': [('background-color', '#003f5c'),
                        ('color', 'white'),
                        ('font-size', '14px'),
                        ('text-align', 'center')]
            },
            {
                'selector': 'tr:nth-child(even)',
                'props': [('background-color', '#f2f2f2')]
            }
        ])
    )

# Affichage du tableau stylé ---
styled_table

```

[114]: <pandas.io.formats.style.Styler at 0x16dd09090>

```

[115]: # Ne garder que les lignes avec une proportion valide (on enlève les NaN pour
↪éviter les soucis d'affichage)
resultat_propre = resultat.dropna(subset=["proportion"]).copy()

# Renommer la colonne "Zone" en "country" pour être cohérent
# avec les autres cartes déjà faites
resultat_propre = resultat_propre.rename(columns={"Zone": "country"})

# Arrondir la proportion
resultat_propre["proportion_arrondie"] = resultat_propre["proportion"].round(2)

# Création de la carte choroplèthe
fig = px.choropleth(
    resultat_propre,
    locations="country",
    locationmode="country names",
    ↪pays
    color="proportion",
    ↪sous-alimentation
    # noms de pays (en français FAO)
    # on précise que ce sont des noms de
    # ce qu'on colore = % de

```

```

    color_continuous_scale="Reds",          # dégradé de rouge = plus c'est foncé,
    ↪ plus c'est grave
    range_color=(0, resultat_propre["proportion"].max()), # échelle 0 → max
    ↪ observé
    hover_name="country",                  # nom du pays au survol
    hover_data={"proportion_arrondie": True, "proportion": False},
    title="Les pays les plus touchés par la sous-alimentation (2017, en %)"
)

# Mise en forme : légende, centrage, marges
fig.update_layout(
    coloraxis_colorbar=dict(
        title="Sous-alimentation<br>(% de la population)"
    ),
    margin=dict(l=20, r=20, t=60, b=20)
)

# Affichage
fig.show()

```

[116]: # 3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013¶

[117]: # Filtrer les données depuis 2013
aide_depuis_2013 = aide_alimentaire[aide_alimentaire['Année'] >= 2013]

[118]: # Calculer le total général d'aide depuis 2013
total_global = aide_depuis_2013["Valeur"].sum()

[119]: # Calculer le total d'aide par pays
aide_totale_par_pays = (aide_depuis_2013.groupby("Zone",
 ↪ as_index=False)["Valeur"].sum().sort_values("Valeur", ascending=False))

[120]: # Calcul de la proportion en %
aide_totale_par_pays["Proportion (%)"] = (aide_totale_par_pays["Valeur"] /
 ↪ total_global * 100).round(2)

[121]: # Extraire le Top 10
top10 = aide_totale_par_pays.head(10)

[122]: # Affichage du tableau
print("Top 10 des pays ayant reçu le plus d'aide alimentaire depuis 2013 (avec
 ↪ proportions) :")
display(top10)

Top 10 des pays ayant reçu le plus d'aide alimentaire depuis 2013 (avec proportions) :

Zone	Valeur	Proportion (%)
------	--------	----------------

50	République arabe syrienne	1858943000	16.84
75	Éthiopie	1381294000	12.52
70	Yémen	1206484000	10.93
61	Soudan du Sud	695248000	6.30
60	Soudan	669784000	6.07
30	Kenya	552836000	5.01
3	Bangladesh	348188000	3.16
59	Somalie	292678000	2.65
53	République démocratique du Congo	288502000	2.61
43	Niger	276344000	2.50

```
[123]: # Tablo désign
styled_top10 = (
    top10.style
    .set_caption("Top 10 des pays ayant reçu le plus d'aide alimentaire depuis_
↳2013")
    .format({"Proportion (%)": "{:.2f}%"})
    .set_properties(**{
        'text-align': 'center',
        'font-size': '14px'
    })
    .set_table_styles([
        {
            'selector': 'th',
            'props': [('background-color', '#003f5c'),
                      ('color', 'white'),
                      ('font-size', '14px'),
                      ('text-align', 'center')]
        },
        {
            'selector': 'tr:nth-child(even)',
            'props': [('background-color', '#f2f2f2')]
        }
    ])
)

styled_top10
```

[123]: <pandas.io.formats.style.Styler at 0x16dc62fd0>

```
[124]: # Création du graphique en barre

# Création d'un dictionnaire comportant 2 listes: Zone et les %
data = {
    "Zone": [
        "République arabe syrienne", "Éthiopie", "Yémen", "Soudan du Sud",
        "Soudan", "Kenya", "Bangladesh", "Somalie",
```

```

        "RDC", "Niger"
    ],
    "Proportion (%)": [16.84, 12.52, 10.93, 6.30, 6.07, 5.01, 3.16, 2.67, 2.61,
↪2.50]
}

df = pd.DataFrame(data)

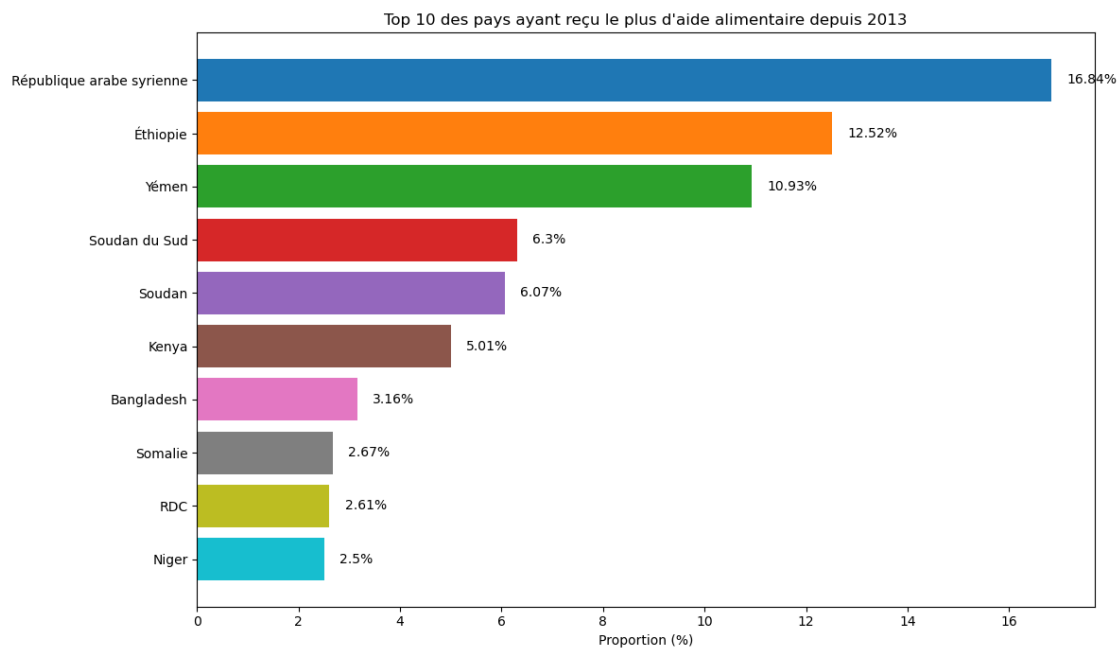
# Correct colormap call without deprecation warning => dde de mettre colour en
↪fonction du drapeau
cmap = plt.colormaps.get_cmap("tab10")
colors = [cmap(i % 10) for i in range(len(df))]

plt.figure(figsize=(12, 7))
plt.barh(df["Zone"], df["Proportion (%)"], color=colors)

# Add proportion labels on bars
for index, value in enumerate(df["Proportion (%)"]):
    plt.text(value + 0.3, index, f"{value}%", va='center')

plt.xlabel("Proportion (%)")
plt.title("Top 10 des pays ayant reçu le plus d'aide alimentaire depuis 2013")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

```



```
[125]: # 3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire
↳ entre 2013 et 2016
```

```
[126]: # Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis
↳ groupby sur zone et année
```

```
[127]: # Filtrer les données entre 2013 et 2016
base = aide_alimentaire.query("2013 <= Année <= 2016")[["Zone", "Année",
↳ "Valeur"]]

display(base.head())
```

	Zone	Année	Valeur
0	Afghanistan	2013	682000
1	Afghanistan	2014	335000
2	Afghanistan	2013	39224000
3	Afghanistan	2014	15160000
4	Afghanistan	2013	40504000

```
[128]: # Calcul du total d'aide alimentaire par pays (2013-2016)
totaux_pays = (base.groupby("Zone", as_index=False)["Valeur"].sum().
↳ sort_values("Valeur", ascending=False))

# Sélection des 5 pays "les plus bénéficiaires"
top5_pays = totaux_pays.head(5)["Zone"].tolist()

print("Les 5 pays qui ont le plus bénéficié de l'aide de 2013-2016 :")
top5_pays
```

Les 5 pays qui ont le plus bénéficié de l'aide de 2013-2016 :

```
[128]: ['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Soudan']
```

```
[129]: # Filtrer la base sur les 5 pays sélectionnés
base_top5 = base[base["Zone"].isin(top5_pays)]

display(base_top5.head())
```

	Zone	Année	Valeur
354	Éthiopie	2013	170000
355	Éthiopie	2014	466000
356	Éthiopie	2015	244000
357	Éthiopie	2013	181066000
358	Éthiopie	2014	178646000

```
[130]: # Calculer l'évolution annuelle des 5 pays
evolution_top5 = (base_top5.groupby(["Zone", "Année"],
↳ as_index=False)["Valeur"].sum().sort_values(["Zone", "Année"]))
```

```
display(evolution_top5)
```

	Zone	Année	Valeur
0	République arabe syrienne	2013	563566000
1	République arabe syrienne	2014	651870000
2	République arabe syrienne	2015	524949000
3	République arabe syrienne	2016	118558000
4	Soudan	2013	330230000
5	Soudan	2014	321904000
6	Soudan	2015	17650000
7	Soudan du Sud	2013	196330000
8	Soudan du Sud	2014	450610000
9	Soudan du Sud	2015	48308000
10	Yémen	2013	264764000
11	Yémen	2014	103840000
12	Yémen	2015	372306000
13	Yémen	2016	465574000
14	Éthiopie	2013	591404000
15	Éthiopie	2014	586624000
16	Éthiopie	2015	203266000

```
[131]: # Transformer en tableau pivot (pays en lignes, années en colonnes)
pivot_top5 = evolution_top5.pivot(index="Zone", columns="Année",
    ↪values="Valeur")

display(pivot_top5)
```

Année	2013	2014	2015	2016
Zone				
République arabe syrienne	563566000.0	651870000.0	524949000.0	118558000.0
Soudan	330230000.0	321904000.0	17650000.0	NaN
Soudan du Sud	196330000.0	450610000.0	48308000.0	NaN
Yémen	264764000.0	103840000.0	372306000.0	465574000.0
Éthiopie	591404000.0	586624000.0	203266000.0	NaN

```
[132]: # Rajouter proportion en vu d'établir courbe

# Totaux mondiaux par année: on regroupe par année
totaux_mondiaux_par_annee = (base.groupby("Année")["Valeur"].sum().to_dict())

df_prop = evolution_top5.copy()

# Ajout des proportions (%) en utilisant les vrais totaux mondiaux
df_prop["Proportion (%)"] = df_prop.apply(lambda row: round((row["Valeur"] /
    ↪totaux_mondiaux_par_annee[row["Année"]]) * 100, 2),axis=1)

display(df_prop)
```

Zone	Année	Valeur	Proportion (%)
------	-------	--------	----------------

0	République arabe syrienne	2013	563566000	13.53
1	République arabe syrienne	2014	651870000	16.55
2	République arabe syrienne	2015	524949000	24.00
3	République arabe syrienne	2016	118558000	15.94
4	Soudan	2013	330230000	7.93
5	Soudan	2014	321904000	8.17
6	Soudan	2015	17650000	0.81
7	Soudan du Sud	2013	196330000	4.71
8	Soudan du Sud	2014	450610000	11.44
9	Soudan du Sud	2015	48308000	2.21
10	Yémen	2013	264764000	6.36
11	Yémen	2014	103840000	2.64
12	Yémen	2015	372306000	17.02
13	Yémen	2016	465574000	62.61
14	Éthiopie	2013	591404000	14.20
15	Éthiopie	2014	586624000	14.89
16	Éthiopie	2015	203266000	9.29

```
[133]: def style_tableau_fao_large(df):

    styler = df.style

    # --- Style des en-têtes ---
    styler = styler.set_table_styles([
        {
            'selector': 'th',
            'props': [
                ('background-color', '#003366'),
                ('color', 'white'),
                ('font-weight', 'bold'),
                ('text-align', 'center'),
                ('padding', '10px'),
                ('font-size', '15px'),
                ('width', '120px')
            ]
        },
        {
            'selector': 'td',
            'props': [
                ('padding', '8px'),
                ('font-size', '14px'),
                ('text-align', 'center'),
                ('border', '1px solid #ddd')
            ]
        }
    ])
    ])
```



```

# --- Ajuster la largeur de la colonne des pays (index) ---
styler = styler.set_properties(
    subset=pd.IndexSlice[:, :1], # sélectionne 1ère colonne
    **{'width': '220px', 'text-align': 'left', 'font-weight': 'bold'}
)

# --- Lignes alternées ---
styler = styler.apply(
    lambda col: [
        'background-color: #f4f6f9' if i % 2 == 0 else 'background-color:
↪white'
        for i in range(len(col))
    ],
    axis=0
)

# --- Formatage des chiffres ---
styler = styler.format("{:.2f}")

return styler

```

```

[134]: pivot_prop = df_prop.pivot(
        index="Zone",
        columns="Année",
        values="Proportion (%)"
    )

pivot_prop.index.name = None # enlève le titre "Zone"
pivot_prop.columns.name = None # enlève le titre "Année"

```

```

[135]: styled_fao = style_tableau_fao_large(pivot_prop)
        styled_fao

```

```

[135]: <pandas.io.formats.style.Styler at 0x16dd0bc50>

```

```

[136]: # Graphique d'évolution des proportions (% du total mondial)

plt.figure(figsize=(10, 6))

for pays in top5_pays:
    data = df_prop[df_prop["Zone"] == pays]
    plt.plot(
        data["Année"],
        data["Proportion (%)"],
        marker='o',
        linewidth=2,
        label=pays
    )

```

```

)

# Titres
plt.title("Évolution de l'aide alimentaire (% du total mondial)\npour les 5_\n↳pays les plus aidés (2013-2016)",
          fontsize=14, fontweight='bold')

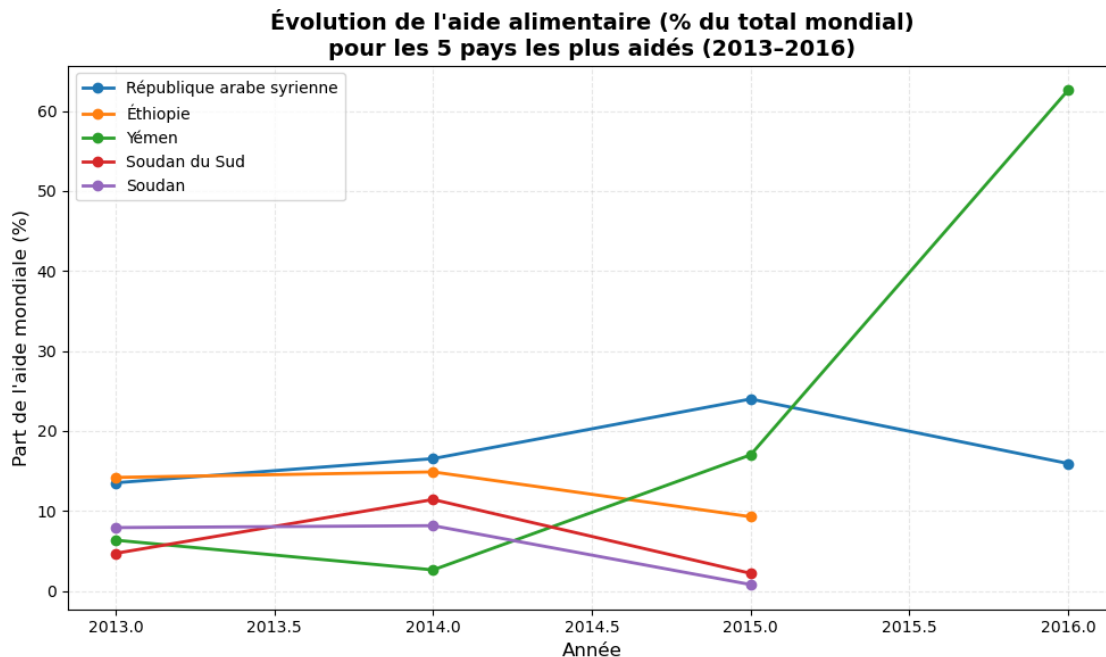
plt.xlabel("Année", fontsize=12)
plt.ylabel("Part de l'aide mondiale (%)", fontsize=12)

# Grille légère
plt.grid(True, which='both', linestyle='--', alpha=0.3)

# Légende
plt.legend(title="", fontsize=10)

# Ajustement
plt.tight_layout()
plt.show()

```



[137]: # Analyse : Évolution annuelle de l'aide alimentaire (2013-2016)

[138]: # Total annuel d'aide alimentaire (tous pays) ==

```

evolution_globale = (base.groupby("Année", as_index=False)["Valeur"].sum().
    ↪sort_values("Année").rename(columns={"Valeur": "Total annuel (kg)"}))

# Calcul de la variation en %
evolution_globale["Variation (%)"] = (evolution_globale["Total annuel (kg)"].
    ↪pct_change() * 100).round(2)

display(evolution_globale)

```

	Année	Total annuel (kg)	Variation (%)
0	2013	4165674000	NaN
1	2014	3939152000	-5.44
2	2015	2187507000	-44.47
3	2016	743568000	-66.01

```

[139]: # .Afficher tableau design
# Formater année ss decimal
evolution_globale["Année"] = evolution_globale["Année"].astype(int)

# faire separat°de millier

def style_tableau_fao_annees(df):

    styler = df.style

    # Style des entêtes
    styler = styler.set_table_styles([
        {
            'selector': 'th',
            'props': [
                ('background-color', '#003366'),
                ('color', 'white'),
                ('font-weight', 'bold'),
                ('text-align', 'center'),
                ('padding', '10px'),
                ('font-size', '15px'),
                ('border', '1px solid #ddd')
            ]
        },
        {
            'selector': 'td',
            'props': [
                ('padding', '8px'),
                ('font-size', '14px'),
                ('text-align', 'center'),
                ('border', '1px solid #ddd')
            ]
        }
    ])

```

```

    ]
    }
])

# Alternance de lignes
styler = styler.apply(
    lambda col: [
        'background-color: #f4f6f9' if i % 2 == 0 else 'background-color:
↪white'
        for i in range(len(col))
    ],
    axis=0
)

styler = styler.format({
    "Année": "{:d}", # année sans décimales
    "Total annuel (kg)": "{:,.0f}", # milliers
    "Variation (%)": lambda x: f"{x:.2f}" if pd.notnull(x) else "" #
↪variation ±
})

return styler

```

```
[140]: display(evolution_globale)
```

	Année	Total annuel (kg)	Variation (%)
0	2013	4165674000	NaN
1	2014	3939152000	-5.44
2	2015	2187507000	-44.47
3	2016	743568000	-66.01

```
[141]: styled_annees = style_tableau_fao_annees(evolution_globale)
        styled_annees
```

```
[141]: <pandas.io.formats.style.Styler at 0x16dd99590>
```

```
[142]: # Graphique courbe

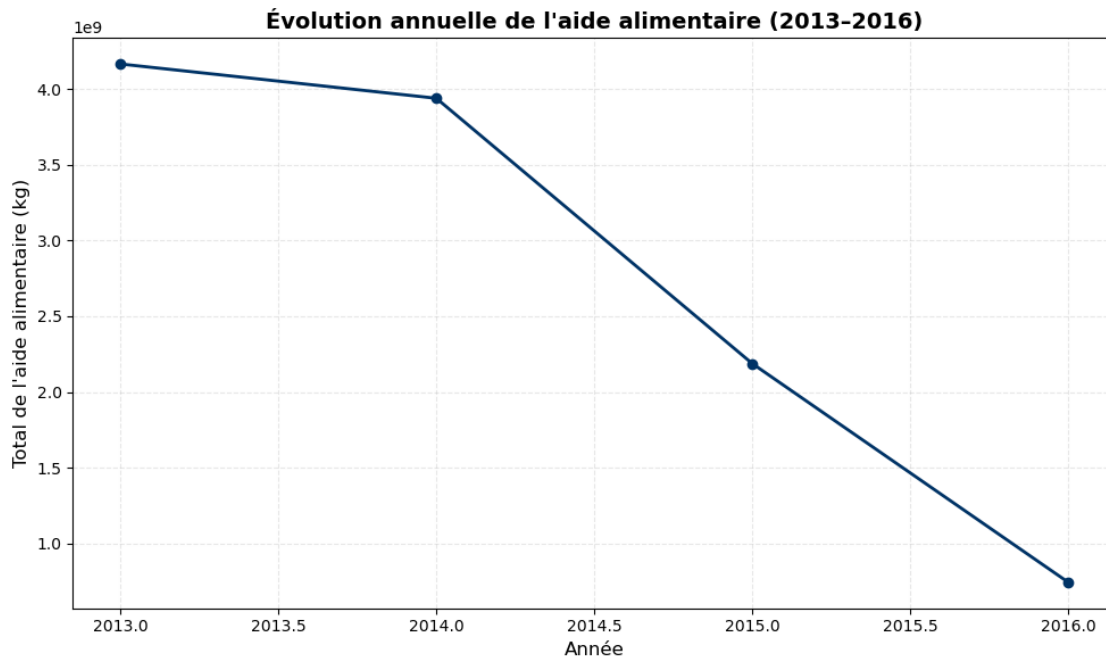
plt.figure(figsize=(10,6))

plt.plot(
    evolution_globale["Année"],
    evolution_globale["Total annuel (kg)"],
    marker="o",
    linewidth=2,
    color="#003366"
)

```

```
plt.title("Évolution annuelle de l'aide alimentaire (2013-2016)",
          fontsize=14, fontweight='bold')
plt.xlabel("Année", fontsize=12)
plt.ylabel("Total de l'aide alimentaire (kg)", fontsize=12)

plt.grid(True, linestyle="--", alpha=0.3)
plt.tight_layout()
plt.show()
```



```
[143]: # Filtrer les données aide alimentaire
aide_produits = aide_alimentaire.query("2013 <= Année <= 2016").copy()

# Regrouper produits avec valeur propre
produits = (aide_produits.groupby("Produit")["Valeur"].sum().reset_index())

# calcul de la somme de tous les produits
total = produits["Valeur"].sum()

# Calcul proportion
produits["Proportion (%)"] = (produits["Valeur"] / total * 100).round(2)

# Select des 2 colonnes Produit + Proportion pour tableau
produits = produits[["Produit", "Proportion (%)"]]
```

```
# Tri décroissant
produits = produits.sort_values("Proportion (%)", ascending=False)

print("Produits les plus distribués (en proportion %) :")
display(produits)
```

Produits les plus distribués (en proportion %) :

	Produit	Proportion (%)
3	Céréales	38.57
1	Blé et Farin	16.14
11	Non-céréales	11.43
4	Céréales Secondaires	10.57
13	Riz, total	6.73
9	Légumineuses Sèches,Tot.	6.68
10	Mélanges et préparations	3.86
7	Huiles végétales	3.13
2	BulgurFarEnt	1.27
14	Sucre, total	0.92
0	Autres non-céréales	0.44
8	Lait, total	0.11
5	Fruits secs, total	0.09
12	Poiss&produi	0.06
6	Graisses Com	0.00
15	Viande Total	0.00

```
[144]: # visualisation design du table
def style_table_products(df):
    return (
        df.style
        .background_gradient(subset=['Proportion (%)'], cmap='Greens')
        .set_table_styles([
            {'selector': 'th', 'props': [
                ('background-color', '#145a32'),
                ('color', 'white'),
                ('font-weight', 'bold'),
                ('text-align', 'center'),
                ('padding', '8px'),
                ('font-size', '14px')
            ]},
            {'selector': 'td', 'props': [
                ('padding', '6px'),
                ('font-size', '13px'),
                ('text-align', 'center')
            ]}
        ])
        .format({'Proportion (%)': '{:.2f}'.format})
    )
```

```

styled_products = style_table_products(produits)
styled_products

```

[144]: <pandas.io.formats.style.Styler at 0x16de34910>

```

[145]: # Affichage des 5 produits slt
top5_produits = produits.head(5)

print("Top 5 des produits les plus distribués (2013-2016) :")
display(top5_produits)

```

Top 5 des produits les plus distribués (2013-2016) :

	Produit	Proportion (%)
3	Céréales	38.57
1	Blé et Farin	16.14
11	Non-céréales	11.43
4	Céréales Secondaires	10.57
13	Riz, total	6.73

```

[146]: # Récupération de ses 5 produits pour visualiser evolution
top5_produits = produits.head(5)["Produit"].tolist()

# Filtrer base pour les Extraire
base_prod5 = aide_produits[aide_produits["Produit"].isin(top5_produits)]

# Calcul de l'evolution annuelle avec proport -Sommes annuelles par produit
evol_prod = (base_prod5.groupby(["Produit", "Année"], as_index=False)["Valeur"].
    ↪sum())

# Totaux mondiaux par année (pour le %)
totaux_annuels = base.groupby("Année")["Valeur"].sum().to_dict()

# Ajout des proportions
evol_prod["Proportion (%)"] = evol_prod.apply(lambda row: round(row["Valeur"] /
    ↪totaux_annuels[row["Année"]] * 100, 2),axis=1)

display(evol_prod)

```

	Produit	Année	Valeur	Proportion (%)
0	Blé et Farin	2013	656791000	15.77
1	Blé et Farin	2014	559060000	14.19
2	Blé et Farin	2015	374904000	17.14
3	Blé et Farin	2016	190760000	25.65
4	Céréales	2013	1589646000	38.16
5	Céréales	2014	1571543000	39.90
6	Céréales	2015	810346000	37.04
7	Céréales	2016	285349000	38.38

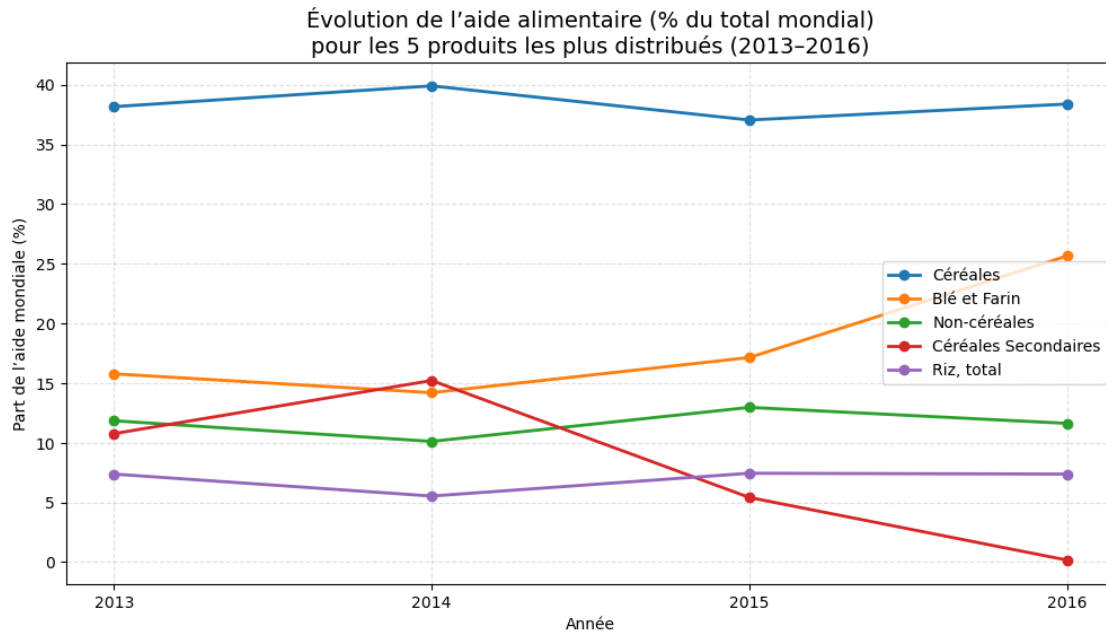
8	Céréales Secondaires	2013	448191000	10.76
9	Céréales Secondaires	2014	598961000	15.21
10	Céréales Secondaires	2015	118235000	5.41
11	Céréales Secondaires	2016	1178000	0.16
12	Non-céréales	2013	493304000	11.84
13	Non-céréales	2014	398314000	10.11
14	Non-céréales	2015	283460000	12.96
15	Non-céréales	2016	86435000	11.62
16	Riz, total	2013	307075000	7.37
17	Riz, total	2014	217823000	5.53
18	Riz, total	2015	162746000	7.44
19	Riz, total	2016	54774000	7.37

```
[147]: plt.figure(figsize=(12, 6))

for prod in top5_produits:
    data = evol_prod[evol_prod["Produit"] == prod]
    plt.plot(data["Année"], data["Proportion (%)"],
             marker='o', linewidth=2, label=prod)

plt.title(
    "Évolution de l'aide alimentaire (% du total mondial)\n"
    "pour les 5 produits les plus distribués (2013-2016)",
    fontsize=14
)

plt.xlabel("Année")
plt.ylabel("Part de l'aide mondiale (%)")
plt.grid(True, linestyle='--', alpha=0.4)
plt.xticks([2013, 2014, 2015, 2016])
plt.legend()
plt.show()
```

[148]: # . 3.9 - Pays avec le moins de disponibilité par habitant

```
[149]: # Calcul de la disponibilité en kcal par personne par jour par pays

# Grouper par pays et sommer la disponibilité alimentaire en kcal
dispo_par_pays = dispo_alimentaire.groupby('Zone')['Disponibilité alimentaire (Kcal/personne/jour)'].sum().reset_index()

# Trier par ordre croissant pour obtenir les pays avec la moins grande disponibilité
dispo_par_pays = dispo_par_pays.sort_values(by='Disponibilité alimentaire (Kcal/personne/jour)')

# Extraire les 10 pays avec la plus faible dispo
top10_moins = dispo_par_pays.head(10)

# Supprimer les décimales
top10_moins.loc[:, 'Disponibilité alimentaire (Kcal/personne/jour)'] = \
    top10_moins['Disponibilité alimentaire (Kcal/personne/jour)'].astype(int)

# Forcer pandas à afficher SANS décimales
pd.set_option('display.float_format', '{:.0f}'.format)

print("Pays avec la moins grande disponibilité alimentaire (Kcal/personne/jour):")
print(top10_moins)
```

Pays avec la moins grande disponibilité alimentaire (Kcal/personne/jour) :

	Zone \
128	République centrafricaine
166	Zambie
91	Madagascar
0	Afghanistan
65	Haïti
133	République populaire démocratique de Corée
151	Tchad
167	Zimbabwe
114	Ouganda
154	Timor-Leste

	Disponibilité alimentaire (Kcal/personne/jour)
128	1879
166	1924
91	2056
0	2087
65	2089
133	2093
151	2109
167	2113
114	2126
154	2129

```
[150]: styled_table = (
    top10_moins.style
    .set_table_styles(
        [{'selector': 'th', 'props': [('background-color', '#0C2D48'),
                                       ('color', 'white'),
                                       ('font-weight', 'bold'),
                                       ('text-align', 'center')]}]
    )
    .set_properties(**{
        'background-color': '#F5F5F5',
        'text-align': 'center'
    })
    .apply(lambda x: ['background-color: #E8EEF1' if i % 2 == 0 else ''
                      for i in range(len(x))], axis=0)
    .format({'Disponibilité alimentaire (Kcal/personne/jour)': '{:.0f}'})
    <# <<< ICI, à l'intérieur !
    .hide(axis="index")
)

styled_table
```

[150]: <pandas.io.formats.style.Styler at 0x16dd98050>

```

[151]: # Préparat° du graphique

top10 = dispo_par_pays.sort_values(by="Disponibilité alimentaire (Kcal/personne/
    ↪jour)", ascending=True).head(10)

# Extraction des données
pays = top10["Zone"]
kcal = top10["Disponibilité alimentaire (Kcal/personne/jour)"]

# Création du graphique

plt.figure(figsize=(10, 6))

# Couleurs du rouge vif → rose pâle
colors = plt.cm.Reds(np.linspace(0.8, 0.2, len(kcal)))

# Barres horizontales
plt.barh(pays, kcal, color=colors)

# Rappel 2000 kcal
plt.axvline(2000, color='black', linestyle='--', linewidth=1.5, label="Seuil : ↪
    ↪2000 kcal/jour")

# Mise en forme du graphique

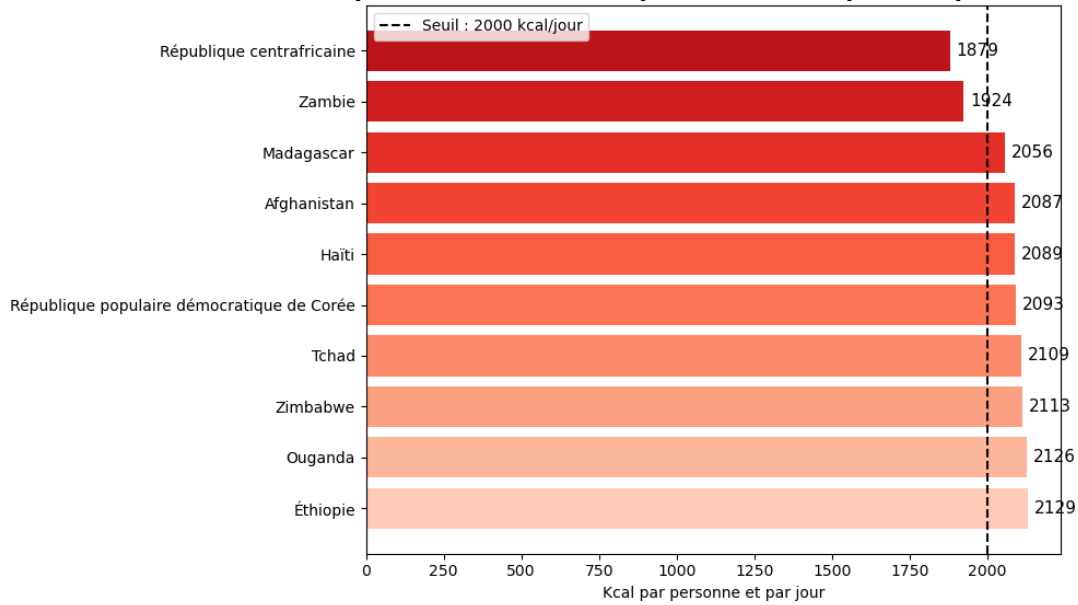
plt.title("Disponibilité alimentaire par habitant (Top 10 les plus faibles)", ↪
    ↪fontsize=15, fontweight='bold')
plt.xlabel("Kcal par personne et par jour")
plt.gca().invert_yaxis() # pays le plus faible en haut

# Ajout des labels numériques sur les barres
for i, v in enumerate(kcal):
    plt.text(v + 20, i, f"{v:.0f}", va='center', fontsize=11)

plt.legend()
plt.tight_layout()
plt.show()

```

Disponibilité alimentaire par habitant (Top 10 les plus faibles)



```
[152]: # 3.10 - Pays avec le plus de disponibilité par habitant
```

```
[153]: # Affichage des 10 pays qui ont le plus de dispo alimentaire par personne

# Grouper par pays et sommer la disponibilité alimentaire en kcal et Trier
# par ordre décroissant pour obtenir les pays avec la plus grande disponibilité

dispo_par_pays = (dispo_alimentaire.groupby('Zone',
    ↪as_index=False)['Disponibilité alimentaire (Kcal/personne/jour)'].sum().
    ↪sort_values(by='Disponibilité alimentaire (Kcal/personne/
    ↪jour)',ascending=False))

top10_dispo = dispo_par_pays.head(10)
print(top10_dispo)
```

	Zone	Disponibilité alimentaire (Kcal/personne/jour)
11	Autriche	3770
16	Belgique	3737
159	Turquie	3708
171	États-Unis d'Amérique	3682
74	Israël	3610
72	Irlande	3602
75	Italie	3578
89	Luxembourg	3540
168	Égypte	3518
4	Allemagne	3503

```
[154]: # Préparat° du graphique
# --- STYLE TABLEAU FAO ---
def style_table_fao_clean(df):
    return (
        df.style
        .set_table_styles([
            {'selector': 'th', 'props': [
                ('background-color', '#003366'),
                ('color', 'white'),
                ('font-weight', 'bold'),
                ('text-align', 'center'),
                ('padding', '8px'),
                ('font-size', '14px'),
            ]},
            {'selector': 'td', 'props': [
                ('padding', '6px'),
                ('font-size', '13px'),
                ('text-align', 'center')
            ]}
        ])
        .format({
            "Disponibilité alimentaire (Kcal/personne/jour)": "{:.0f}"
        })
        .apply(
            lambda col: [
                'background-color: #f0f8ff' if i % 2 == 0 else
                ↪ 'background-color: white'
                for i in range(len(col))
            ],
            axis=0
        )
    )

# --- APPLICATION DU STYLE (TOP 10) ---
top10_clean = top10_dispo[['Zone', 'Disponibilité alimentaire (Kcal/personne/
↪ jour)']]
styled_top10 = style_table_fao_clean(top10_clean)

styled_top10
```

[154]: <pandas.io.formats.style.Styler at 0x16dc62d50>

```
[155]: # 1) préparation des Données (TOP 10) pour graphique
# =====
top10 = dispo_par_pays.sort_values(
    by="Disponibilité alimentaire (Kcal/personne/jour)",
    ascending=False
```

```

).head(10)

pays = top10["Zone"]
kcal = top10["Disponibilité alimentaire (Kcal/personne/jour)"]

# =====
# 2) Graphique
# =====
plt.figure(figsize=(10, 6))

# Dégradé vert FAO : foncé → clair
colors = plt.cm.Greens(np.linspace(0.9, 0.3, len(kcal)))

plt.barh(pays, kcal, color=colors)

# Ligne seuil FAO (modifiable)
seuil = 2500
plt.axvline(
    seuil,
    color='black',
    linestyle='--',
    linewidth=1.5,
    label=f"Seuil FAO : {seuil} kcal/jour"
)

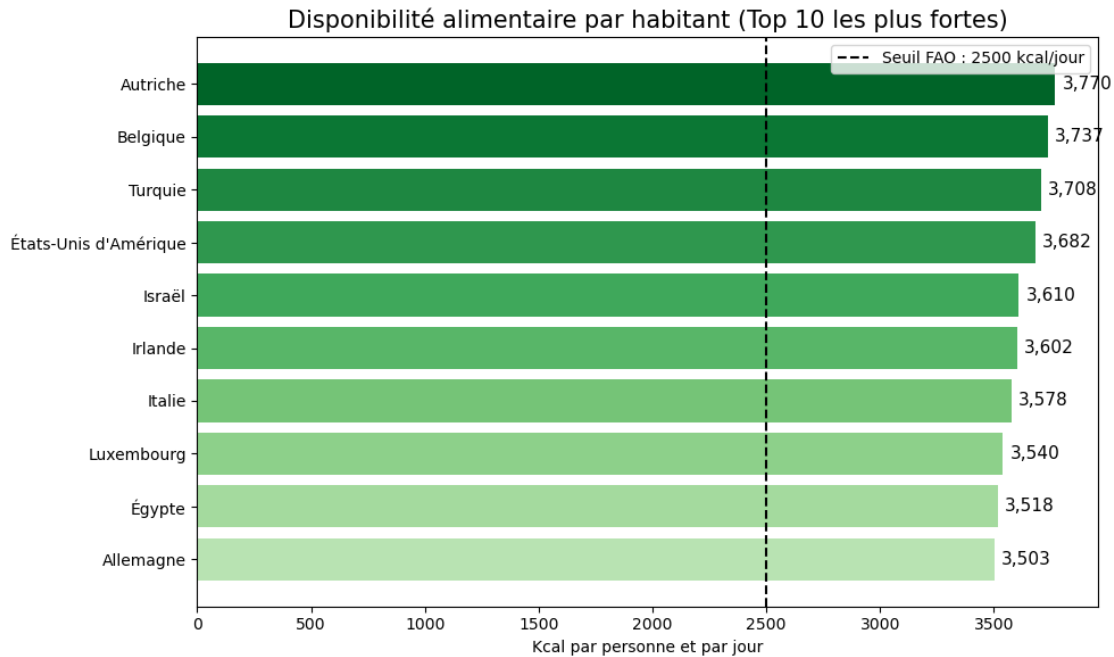
# Inverser l'axe (pour avoir le top en haut)
plt.gca().invert_yaxis()

# Titres
plt.title("Disponibilité alimentaire par habitant (Top 10 les plus fortes)",
          fontsize=15)
plt.xlabel("Kcal par personne et par jour")

# Ajouter les valeurs au bout des barres
for i, v in enumerate(kcal):
    plt.text(v + 30, i, f"{v:,.0f}", va='center', fontsize=11)

plt.legend()
plt.tight_layout()
plt.show()

```



```
[156]: dispo_par_pays = dispo_alimentaire.groupby("Zone",
        ↪as_index=False)["Disponibilité alimentaire (Kcal/personne/jour)"].sum()

# Renommer pour simplifier
dispo_par_pays = dispo_par_pays.rename(columns={"Disponibilité alimentaire (Kcal/personne/jour)": "dispo_kcal"})
```

```
[157]: # Préparation des données
dispo_par_pays = dispo_alimentaire.groupby("Zone", as_index=False)[
    "Disponibilité alimentaire (Kcal/personne/jour)"
].sum()

# Renommer pour simplifier
dispo_par_pays = dispo_par_pays.rename(columns={
    "Disponibilité alimentaire (Kcal/personne/jour)": "dispo_kcal",
    "Zone": "country"    # très important pour la suite !
})

# 2) Mapping FAO → noms reconnus par Plotly
mapping_manual = {
    "États-Unis d'Amérique": "United States",
    "Fédération de Russie": "Russia",
    "Iran (République islamique d')": "Iran",
    "République de Corée": "South Korea",
    "République populaire démocratique de Corée": "North Korea",
```

```

    "République-Unie de Tanzanie": "Tanzania",
    "Venezuela (République bolivarienne du)": "Venezuela",
    "Côte d'Ivoire": "Ivory Coast",
    "Égypte": "Egypt",
    "Chine, continentale": "China",
    "Chine - RAS de Hong-Kong": "Hong Kong",
    "Chine - RAS de Macao": "Macao",
    "Chine, Taiwan Province de": "Taiwan",
    "République démocratique populaire lao": "Laos",
    "République centrafricaine": "Central African Republic",
    "République dominicaine": "Dominican Republic",
    "République de Moldova": "Moldova",
    "Viet Nam": "Vietnam",
    "Eswatini": "Eswatini",
    "Haïti": "Haiti",
    "République arabe syrienne": "Syria",
    "République du Yémen": "Yemen",
    "Soudan": "Sudan",
    "Soudan du Sud": "South Sudan",
    "Tchad": "Chad",
    "Pays-Bas": "Netherlands",
    "Royaume-Uni": "United Kingdom",
    "Afrique du Sud": "South Africa"
}

dispo_par_pays["country_plotly"] = dispo_par_pays["country"].
↳replace(mapping_manual)

# 3) Palette rouge → vert
custom_colors = [
    "#800000", "#FF4500", "#FFA500",
    "#FFFF66", "#ADFF2F", "#006400"
]

# 4) Création de la carte
fig = px.choropleth(
    dispo_par_pays,
    locations="country_plotly",
    hover_name="country",
    locationmode="country names",
    color="dispo_kcal",
    color_continuous_scale=custom_colors,
    title=" Disponibilité alimentaire (kcal/personne/jour)",
)

fig.update_geos(
    projection_type="natural earth",

```



```

        center=dict(lat=10, lon=0),
        projection_scale=1.05,
        fitbounds="locations"
    )

    fig.update_layout(
        width=1300,
        height=750,
        margin=dict(l=20, r=20, t=60, b=100),
        coloraxis_colorbar=dict(
            title="kcal / pers / jour",
            orientation="h",
            thickness=15,
            len=0.70,
            x=0.5,
            xanchor="center",
            y=-0.15
        )
    )

    fig.show()

```

[158]: # 3.11 - Exemple de la Thaïlande pour le Manioc

[159]: # création d'un dataframe avec uniquement la Thaïlande

```

# On filtre de chq df la Thaïlande et le manioc
df_population_thai = population[population["Zone"] == "Thaïlande"]
df_sous_nutrition_thai = sous_nutrition[sous_nutrition["Zone"] == "Thaïlande"]
df_dispo_alimentaire_thai = dispo_alimentaire[(dispo_alimentaire["Zone"] ==
↳ "Thaïlande")]

```

[160]: # . on relie chaque df ensemble

```

df_thaïlande = df_population_thai.
↳ merge(df_dispo_alimentaire_thai,on=["Zone"],how="left")

print(df_thaïlande[["Zone", "Année", "Population",
↳ "Produit", "Origine", "Disponibilité intérieure", "Production", "Nourriture"]].
↳ head())

```

	Zone	Année	Population	Produit	Origine \
0	Thaïlande	2013	68144518	Abats Comestible	animale
1	Thaïlande	2013	68144518	Agrumes, Autres	vegetale
2	Thaïlande	2013	68144518	Alcool, non Comestible	vegetale
3	Thaïlande	2013	68144518	Aliments pour enfants	vegetale
4	Thaïlande	2013	68144518	Ananas	vegetale

	Disponibilité intérieure	Production	Nourriture
0	74000000	45000000	75000000
1	8000000	12000000	6000000
2	358000000	447000000	0
3	12000000	0	12000000
4	782000000	2209000000	671000000

```
[161]: # Dictionnaire de transformation
correspondance = {
    "2012-2014": 2013,
    "2013-2015": 2014,
    "2014-2016": 2015,
    "2015-2017": 2016,
    "2016-2018": 2017,
    "2017-2019": 2018
}

# Filtrer pour la Thaïlande
df_thai_sous_nutrition = sous_nutrition[sous_nutrition["Zone"] == "Thaïlande"].
↳copy()

# Transformer la colonne Année
df_thai_sous_nutrition["Année"] = df_thai_sous_nutrition["Année"].
↳map(correspondance)

# Vérifier
print(df_thai_sous_nutrition[["Zone", "Année", "sous_nutrition"]].head())
```

	Zone	Année	sous_nutrition
1110	Thaïlande	2013	6200000
1111	Thaïlande	2014	6000000
1112	Thaïlande	2015	5900000
1113	Thaïlande	2016	6000000
1114	Thaïlande	2017	6200000

```
[162]: # Création du df final df_thaïlande

df_thaïlande = df_thaïlande.merge(df_thai_sous_nutrition,on=["Zone", "Année"],
↳how="left")
```

```
[163]: # Choix des Colonnes clés à visualiser

colonnes_importantes = [
    "Année",
    "Zone",
    "Population",
    "sous_nutrition",
```

```

    "Produit",
    "Origine",
    "Disponibilité intérieure",
    "Exportations - Quantité",
    "Importations - Quantité",
    "Nourriture",
    "Pertes",
    "Production",
    "Semences",
    "Traitement",
    "Variation de stock"
]

display(df_thailande[colonnes_importantes].head())

```

	Année	Zone	Population	sous_nutrition	Produit \
0	2013	Thaïlande	68144518	6200000	Abats Comestible
1	2013	Thaïlande	68144518	6200000	Agrumes, Autres
2	2013	Thaïlande	68144518	6200000	Alcool, non Comestible
3	2013	Thaïlande	68144518	6200000	Aliments pour enfants
4	2013	Thaïlande	68144518	6200000	Ananas

	Origine	Disponibilité intérieure	Exportations - Quantité \
0	animale	74000000	5000000
1	végétale	8000000	6000000
2	végétale	358000000	110000000
3	végétale	12000000	7000000
4	végétale	782000000	1449000000

	Importations - Quantité	Nourriture	Pertes	Production	Semences \
0	33000000	75000000	0	45000000	0
1	2000000	6000000	0	12000000	0
2	21000000	0	0	447000000	0
3	19000000	12000000	0	0	0
4	9000000	671000000	110000000	2209000000	0

	Traitement	Variation de stock
0	0	0
1	2000000	0
2	0	0
3	0	0
4	0	13000000

```
[164]: # Calcul de la sous nutrition en Thaïlande
```

```
[165]: # Création du tableau sous nutrition directement depuis df_thailande
```

```

tableau_sous_nutrition = df_thailande[["Année", "Population",
↪ "sous_nutrition"]].copy()

display(tableau_sous_nutrition)

```

	Année	Population	sous_nutrition
0	2013	68144518	6200000
1	2013	68144518	6200000
2	2013	68144518	6200000
3	2013	68144518	6200000
4	2013	68144518	6200000
..	...	...	...
565	2018	69428453	6500000
566	2018	69428453	6500000
567	2018	69428453	6500000
568	2018	69428453	6500000
569	2018	69428453	6500000

[570 rows x 3 columns]

[166]: *# Rajouter la colonne Taux_sous_nutrition*

```

tableau_sous_nutrition["Taux_sous_nutrition (%)"] =
↪ (tableau_sous_nutrition["sous_nutrition"].astype(float) /
↪ tableau_sous_nutrition["Population"].astype(float) * 100).round(2)

# Garder seulement une ligne unique par année
tableau_sous_nutrition = tableau_sous_nutrition.
↪ drop_duplicates(subset=["Année"]).reset_index(drop=True)

```

[167]: *# STYLE FAO pour tableau_sous_nutrition (version optimisée) pour Power Point*

```

# Formater les nombres
df_style_fao = (
    tableau_sous_nutrition.style
    .format({
        "Année": "{:.0f}".format,
        "Population": lambda x: f"{x:,.0f}".replace(",", " "),
        "sous_nutrition": lambda x: f"{x:,.0f}".replace(",", " "),
        "Taux_sous_nutrition (%)": "{:.2f}%".format
    })
    .set_properties(**{ # on peint les cases
        "background-color": "#e8f5e9", # Vert très clair FAO
        "border": "1px solid #999",
        "padding": "6px",
        "text-align": "center"
    })
)

```

```

.set_table_styles([# on fait de joli en-têtes
    {"selector": "th",
     "props": [("background-color", "#00796b"), # Vert foncé FAO
                ("color", "white"),
                ("padding", "8px"),
                ("font-weight", "bold"),
                ("text-align", "center")]}
    ])
.set_caption("Thaïlande - Évolution de la sous-nutrition (2013-2018)")
)

# Afficher

display(df_style_fao)

```

<pandas.io.formats.style.Styler at 0x16fdf4050>

```

[168]: # Filtrer les années 2013-2017 et trier
df_graph = tableau_sous_nutrition[tableau_sous_nutrition['Année'] <= 2017].
    ↪copy()
df_graph = df_graph.sort_values('Année')

années = df_graph['Année'].tolist()
population = df_graph['Population'].tolist()
taux = df_graph['Taux_sous_nutrition (%)'].tolist()

fig, ax1 = plt.subplots(figsize=(12, 7))

# Barres de population
ax1.bar(années, population, color='#87CEEB', alpha=0.7, width=0.6)
ax1.set_xlabel('Année', fontsize=14, fontweight='bold')
ax1.set_ylabel('Population (en millions)', fontsize=14, fontweight='bold',
    ↪color='#1E90FF')
ax1.tick_params(axis='y', labelcolor='#1E90FF')
ax1.set_ylim(65_000_000, max(population) * 1.05)
ax1.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'{x/1_000_000:.
    ↪1f}M'))

# Ligne du taux
ax2 = ax1.twinx()
ax2.plot(années, taux, marker='o', linewidth=3, color='#087965', markersize=10)
ax2.set_ylabel('Taux de sous-nutrition (%)', fontsize=14, fontweight='bold',
    ↪color='#087965')
ax2.tick_params(axis='y', labelcolor='#087965')

# Annotations
for année, pop, t in zip(années, population, taux):

```

```

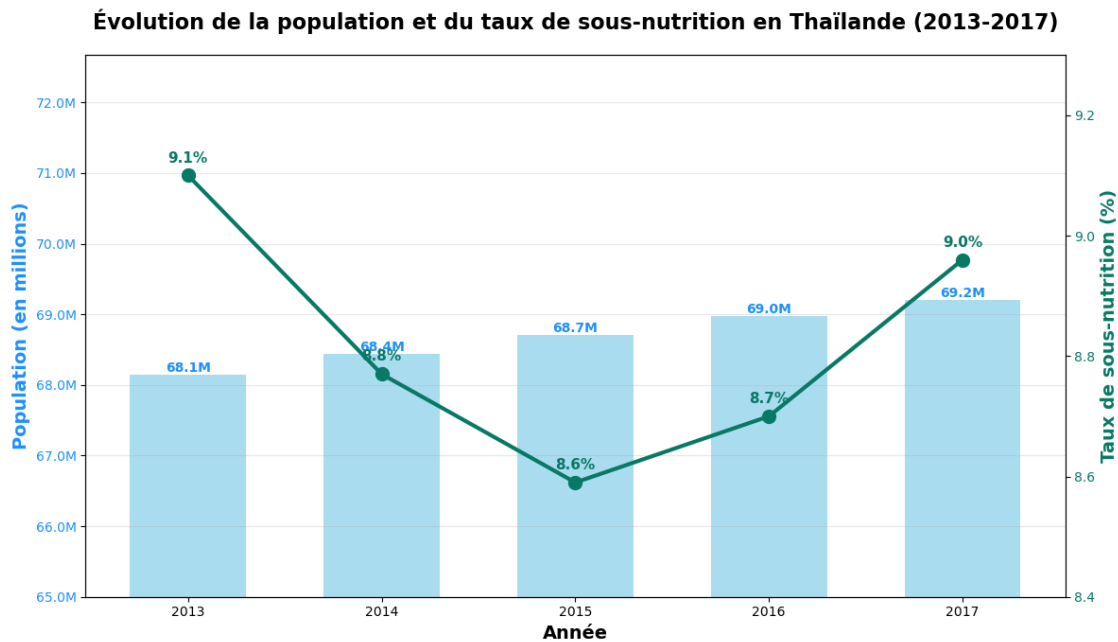
# Population
ax1.text(année, pop, f'{pop/1_000_000:.1f}M',
         ha='center', va='bottom', fontsize=10, fontweight='bold',
         color='#1E90FF')

# Taux
ax2.annotate(f'{t:.1f}%', xy=(année, t), xytext=(0, 10),
            textcoords='offset points', ha='center',
            fontsize=11, fontweight='bold', color='#087965')

ax1.set_xticks(années)
ax1.grid(True, alpha=0.3, axis='y')
ax2.set_ylim(8.4, 9.3)

plt.title('Évolution de la population et du taux de sous-nutrition en Thaïlande,
         (2013-2017)',
         fontsize=16, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

```



```

[169]: # Calculer la disponibilité alimentaire de la Thaïlande
somme_dispo_2017 = df_thaïlande[df_thaïlande["Année"] == 2017]["Disponibilité_
         alimentaire (Kcal/personne/jour)"].sum()

print(f"Disponibilité alimentaire totale en Thaïlande en 2017 :_
      {somme_dispo_2017:.0f} kcal/personne/jour")

```

Disponibilité alimentaire totale en Thaïlande en 2017 : 2785 kcal/personne/jour

```
[170]: # Formater le nombre avec espace comme séparateur (style français)
```

```
nombre_formate = f"{somme_dispo_2017:,.0f}".replace(",", " ")
```

```
carte = f"""
```

```
    THAÏLANDE - 2017
```

```
    DISPONIBILITÉ ALIMENTAIRE TOTALE
```

```
        {nombre_formate:>9s}
        kcal/pers/jour
```

```
    """
```

```
print(carte)
```

```
    THAÏLANDE - 2017
```

```
    DISPONIBILITÉ ALIMENTAIRE TOTALE
```

```
        2 785
        kcal/pers/jour
```

```
[171]: # On calcule la proportion exportée en fonction de la proportion - Manioc
```

```
[172]: # Filtrer uniquement le manioc en Thaïlande pour 2017
```

```
df_thaïlande_manioc_2017 = df_thaïlande[(df_thaïlande["Produit"] == "Manioc") &
    ↪(df_thaïlande["Année"] == 2017)].copy()
```

```
display(df_thaïlande_manioc_2017)
```

	Zone	Année	Population	Produit	Origine	Aliments pour animaux \
430	Thaïlande	2017	69209810	Manioc	vegetale	1800000000

Autres Utilisations Disponibilité alimentaire (Kcal/personne/jour) \

```

430          2081000000          40

Disponibilité alimentaire en quantité (kg/personne/an) \
430          13

Disponibilité de matière grasse en quantité (g/personne/jour) ... \
430          0          ...

Disponibilité intérieure Exportations - Quantité \
430          6264000000          25214000000

Importations - Quantité Nourriture Pertes Production Semences \
430          1250000000 871000000 1511000000 30228000000 0

Traitement Variation de stock sous_nutrition
430          0          0          6200000

[1 rows x 21 columns]

```

```

[173]: # Mets juste cette ligne et donne-moi le résultat :
print(df_thailande_manioc_2017["Disponibilité intérieure"].values[0])

```

```
6264000000.0
```

```

[174]: # J'identifie chq valeur des colonnes que j'ai besoins
production = df_thailande_manioc_2017["Production"].values[0]
exportations = df_thailande_manioc_2017["Exportations - Quantité"].values[0]
importations = df_thailande_manioc_2017["Importations - Quantité"].values[0]
disponibilite = df_thailande_manioc_2017["Disponibilité intérieure"].values[0]

# afficher en travaillant visuel pour visualiser
↳ Disponibilité = Production + Importations - Exportations
print("=== DONNÉES DU MANIOC 2017 ===")
print(f"Production:      {production:>15,.0f} Kg".replace(","," ")) # :>15
↳ aligne à droite sur 15 caractères
print(f"Exportations:    {exportations:>15,.0f} Kg".replace(","," "))
print(f"Importations:    {importations:>15,.0f} Kg".replace(","," "))
print("-" * 40)
print(f"Disponibilité:    {disponibilite:>15,.0f} Kg".replace(","," "))
print("(Production + Importations - Exportations)")
print()

```

```

=== DONNÉES DU MANIOC 2017 ===
Production:      30 228 000 000 Kg
Exportations:    25 214 000 000 Kg
Importations:    1 250 000 000 Kg
-----
Disponibilité:    6 264 000 000 Kg

```


(Production + Importations - Exportations)

```
[175]: # calculer % des flux : entrées/sorties

total_flux = production + importations # Tout ce qui entre
proportion_production = (production / total_flux) * 100
proportion_importations = (importations / total_flux) * 100

print("=== PROPORTIONS DES FLUX ===")
print(f"Production:    {proportion_production:.1f}% ")
print(f"Importations: {proportion_importations:.1f}% ")
print(f"Exportations: {exportations/production*100:.1f}% ")
print()

=== PROPORTIONS DES FLUX ===
Production:    96.0%
Importations:  4.0%
Exportations:  83.4%

[176]: # J'identifie chq valeur des colonnes que j'ai besoins
nourriture = df_thailande_manioc_2017["Nourriture"].values[0]
pertes = df_thailande_manioc_2017["Pertes"].values[0]
disponibilite = df_thailande_manioc_2017["Disponibilité intérieure"].values[0]

# Calcul du reste (autres utilisations : semences, traitement, etc.)
autres_usages = disponibilite - (nourriture + pertes)

# Afficher calcul comment est utilise la Disponibilité
print("=== UTILISATION DE LA DISPONIBILITÉ ===")
print(f"Nourriture:    {nourriture:>15,.0f} Kg".replace(","," "))
print(f"Pertes:        {pertes:>15,.0f} Kg".replace(","," "))
print(f"Autres usages*:  {autres_usages:>15,.0f} Kg".replace(","," "))
print(f" (semences, traitement, variation de stock)")
print()

=== UTILISATION DE LA DISPONIBILITÉ ===
Nourriture:    871 000 000 Kg
Pertes:        1 511 000 000 Kg
Autres usages*: 3 882 000 000 Kg
 (semences, traitement, variation de stock)
```

```
[177]: # calculer % de l'utilisation de la disponibilité
print("=== RÉPARTITION DE LA DISPONIBILITÉ ===")
utilisation_data = {
    "Nourriture": nourriture,
```

```

    "Pertes": pertes,
    "Autres usages": autres_usages
}

for usage, valeur in utilisation_data.items():
    pourcentage = (valeur / disponibilite) * 100
    print(f"{usage}: {pourcentage:.1f}%")

```

=== RÉPARTITION DE LA DISPONIBILITÉ ===

Nourriture: 13.9%

Pertes: 24.1%

Autres usages: 62.0%

```

[178]: # === GRAPHIQUE 1 : LES FLUX ===
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))

# Données pour le graphique des flux
flux_labels = ["Production", "Exportations", "Importations"]
flux_values = [production, exportations, importations]

# Pourcentages (calculés différemment)
flux_pourcentages = [
    96.0, # Production = 96% de ce qui entre
    83.4, # Exportations = 83.4% de la production
    4.0   # Importations = 4% de ce qui entre
]

flux_colors = ["#3E6F3E", "#A83232", "#1E3A8A"]

# Graphique en barres pour les flux
bars = ax1.bar(flux_labels, flux_values, color=flux_colors, edgecolor='black')
ax1.set_title("Flux du Manioc en Thaïlande (2017)", fontsize=14,
    ↪fontweight='bold', pad=20)
ax1.set_ylabel("kg", fontsize=12)

# Comme vos valeurs sont grandes, on affiche en notation scientifique autour de
    ↪1e9 kg (Md kg)
ax1.ticklabel_format(axis='y', style='sci', scilimits=(9, 9))
ax1.grid(axis='y', alpha=0.3)

# Ajouter les valeurs sur les barres (en Md kg)
for bar, valeur, pourcentage in zip(bars, flux_values, flux_pourcentages):
    height = bar.get_height()
    ax1.text(
        bar.get_x() + bar.get_width()/2., height * 1.02,
        f'{valeur/1e9:.2f} Md kg\n({pourcentage}%)',
        ha='center', va='bottom', fontsize=10
    )

```

```

)

ax1.set_ylim(0, max(flux_values) * 1.15)

# === GRAPHIQUE 2 : RÉPARTITION DE LA DISPONIBILITÉ ===
dispo_labels = ["Nourriture", "Pertres", "Autres usages"]
dispo_values = [nourriture, pertes, autres_usages]
dispo_pourcentages = [13.9, 24.1, 62.0]
dispo_colors = ["#FB8C00", "#D9DFE2", "#4A90E2"]

wedges, texts, autotexts = ax2.pie(
    dispo_values,
    labels=dispo_labels,
    autopct='%1.1f%%',
    colors=dispo_colors,
    startangle=90,
    wedgeprops={'edgecolor': "white", "linewidth": 2},
    textprops={'fontsize': 11}
)

# Titre cohérent en kg (6,3 Md kg si dispo_total 6.3e9)
ax2.set_title("Répartition de la Disponibilité\n(6,3 Md kg)",
              fontsize=14, fontweight='bold', pad=20)

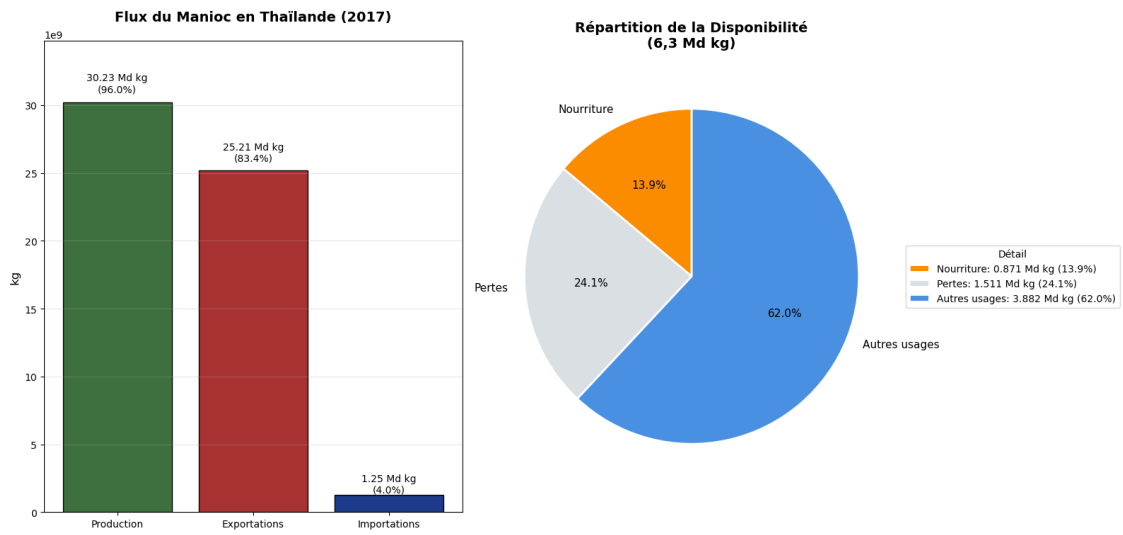
# Légende cohérente en Md kg
legende_texts = []
for label, valeur, pourcentage in zip(dispo_labels, dispo_values,
    ↪dispo_pourcentages):
    legende_texts.append(f"{label}: {valeur/1e9:.3f} Md kg ({pourcentage}%)")

ax2.legend(wedges, legende_texts, title="Détail", loc="center left",
           bbox_to_anchor=(1, 0, 0.5, 1), fontsize=10)

plt.suptitle("ANALYSE DU MANIOC EN THAÏLANDE - 2017", fontsize=16,
    ↪fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

ANALYSE DU MANIOC EN THAÏLANDE - 2017



[]:

[]: